TRƯỜNG ĐẠI HỌC VĂN LANG · KHOA CÔNG NGHỆ THÔNG TIN

CHUYÊN ĐỀ TỐT NGHIỆP 1 · HỌC KỲ 1, NĂM HỌC 2026 – 2027

Tài liệu tự học buổi 7 — Hiện thực module cốt lõi theo chuyên ngành

Tách lớp và CRUD trên CSDL quan hệ [SE] · Pipeline ETL có kiểm chất lượng và chạy lại được [DA] · Thu thập, làm sạch và chia dữ liệu đúng cách [AI]. Mỗi phần có mã mẫu hoàn chỉnh chạy được trên bộ dữ liệu Mekong Mobile.

Sản phẩm của tài liệu này là thành phần 1 và 2 của Bài tập 2 — module cốt lõi chạy được và bộ kiểm thử của nó. Bắt đầu ở workshop buổi 8, nộp cuối buổi 12.

Biên soạn: **TS. Nguyễn Trí Hải** · Bộ môn Kỹ thuật phần mềm

Tài liệu lưu hành nội bộ · Bộ tài liệu tự học phần chuyên ngành (30% thời lượng lý thuyết buổi 7).

Tài liệu song ngữ Việt – Anh: bản tiếng Anh (English version) nằm ngay sau Phần D của bản tiếng Việt.

Cách dùng tài liệu này

Giảng viên giảng trên lớp (buổi 7)	Sinh viên tự đọc trong tài liệu này
Module cốt lõi là gì và cách chọn nó; nguyên tắc tách lớp và phụ thuộc một chiều; ba tầng kiểm tra dữ liệu vào; ý nghĩa của "chạy được bằng một lệnh"; kỷ luật commit và thông điệp commit; nguyên tắc viết test đầu tiên; bộ ba câu hỏi chuẩn áp dụng cho code.	Cách hiện thực cụ thể trên stack của track mình: cấu trúc thư mục, mã mẫu từng lớp, cách viết test, và checklist tự kiểm trước khi tới workshop buổi 8.

Bốn điều đúng cho cả ba track — đọc trước khi vào phần của mình

1. TÁCH LỚP là yêu cầu bắt buộc, không phải khuyến nghị. Mọi track đều phải có ít nhất ba lớp tách biệt: lớp nhận đầu vào · lớp xử lý nghiệp vụ · lớp truy cập dữ liệu. Dồn tất cả vào một tệp là lỗi bị hạ mức rubric ở tiêu chí "Chất lượng kỹ thuật" (50% điểm bài tập).

2. CHẠY ĐƯỢC BẰNG MỘT LỆNH. Không phải "mở notebook rồi bấm từng ô", không phải "chạy tệp này rồi chạy tệp kia". Một lệnh, ghi trong README, chạy được trên máy khác.

3. CHẠY LẠI ĐƯỢC MÀ KHÔNG HỎNG DỮ LIỆU. Chạy lần thứ hai phải cho kết quả giống lần thứ nhất, không nhân đôi bản ghi, không lỗi vì "bảng đã tồn tại". Đây là mục kiểm chứng của Bài tập 2.

4. TEST CHỨNG MINH ĐƯỢC CODE CHẠY ĐÚNG. Ít nhất ba test cho phần nghiệp vụ quan trọng nhất, trong đó có một test cho trường hợp dữ liệu sai. Test phải chạy được bằng một lệnh và phải PASS khi giảng viên chạy.

Nhắc lại phạm vi module cốt lõi của Bài tập 2

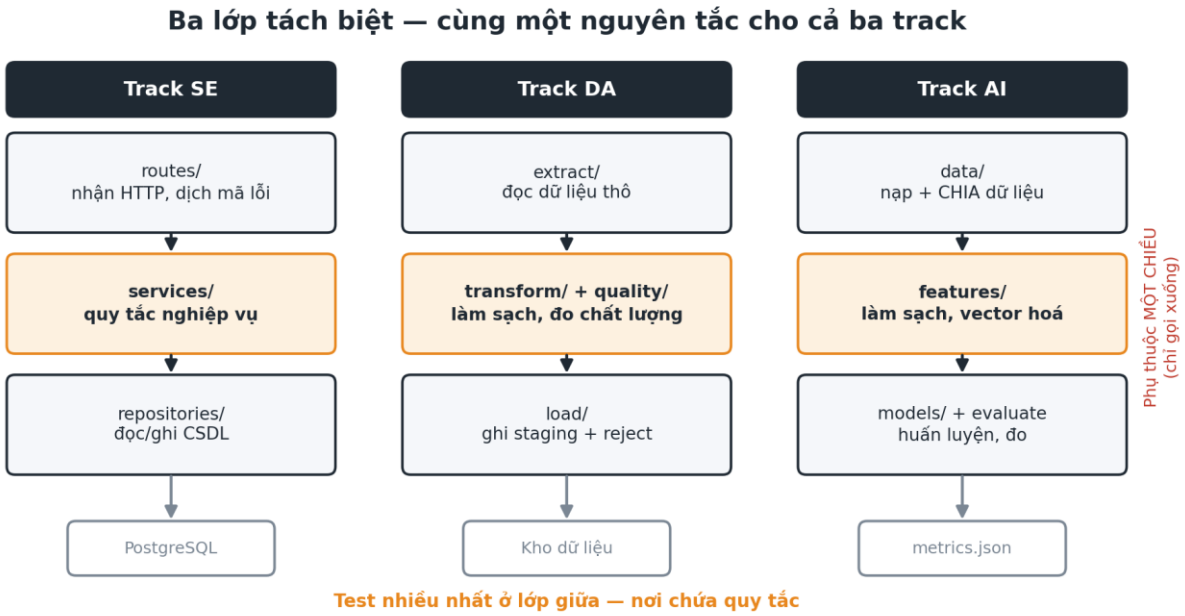
Module cốt lõi là phần hệ thống mà nếu bỏ đi thì sản phẩm không còn ý nghĩa. Nó KHÔNG phải là phần đẹp nhất, cũng không phải là phần nhiều code nhất. Với case study Mekong Mobile, ba lựa chọn module cốt lõi tiêu biểu theo track là:

Track	Module cốt lõi tiêu biểu	Luồng nghiệp vụ tương ứng	Vì sao nó là cốt lõi
SE	Tiếp nhận và xử lý phiếu bảo hành	L2 — tiếp nhận thiết bị, kiểm tra hạn bảo hành, phân công kỹ thuật viên	Bỏ nó đi thì hệ thống không còn phục vụ được nghiệp vụ chính của trung tâm bảo hành.
DA	Pipeline làm sạch và hợp nhất dữ liệu khách hàng	L6 — hợp nhất khách hàng trùng lặp từ nhiều cửa hàng	Mọi báo cáo phía sau đều dựa trên dữ liệu khách hàng đã làm sạch; sai ở đây thì mọi thứ phía sau đều sai.
AI	Phân loại tự động mô tả sự cố thành nhóm lỗi	L10 — tự động gợi ý nhóm lỗi khi nhân viên nhập mô tả	Bỏ nó đi thì sản phẩm chỉ còn là một biểu mẫu nhập tay, không còn giá trị AI nào.

Nếu module cốt lõi em chọn ở Bài tập 1 khác ba ví dụ trên, vẫn giữ nguyên lựa chọn của em. Ba ví dụ này chỉ là mốc so sánh về mức độ và phạm vi.

Minh hoạ: ba lớp của ba track trong một hình

Trước khi đọc phần của track mình, hãy nhìn hình dưới. Cả ba track có cùng một hình dạng: lớp trên nhận đầu vào, lớp giữa chứa quy tắc, lớp dưới đọc ghi dữ liệu. Mũi tên chỉ đi xuống — lớp dưới không bao giờ gọi ngược lên lớp trên.



Hình 1. Lớp giữa (tô cam) là nơi chứa quy tắc nghiệp vụ và là nơi cần test nhiều nhất. Nếu em không chỉ ra được lớp giữa của mình nằm ở thư mục nào, cấu trúc chưa đạt.

PHẦN A — Track SE: Tách lớp và CRUD trên cơ sở dữ liệu quan hệ

A1. Cấu trúc thư mục bắt buộc

Cấu trúc dưới đây là cấu trúc tối thiểu được chấp nhận cho Bài tập 2. Tên thư mục có thể đổi theo quy ước của framework em dùng, nhưng SỰ TÁCH BIỆT giữa bốn tầng thì không được bỏ.

Cấu trúc thư mục — ví dụ với Node.js + Express (Spring Boot tương tự)

```
du-an/
├── src/
│   ├── routes/              # TẦNG 1: nhận HTTP, không có logic nghiệp vụ
│   │   └── ticket.routes.js
│   ├── services/            # TẦNG 2: logic nghiệp vụ, KHÔNG biết gì về HTTP
│   │   └── ticket.service.js
│   ├── repositories/        # TẦNG 3: truy cập CSDL, KHÔNG biết gì về nghiệp vụ
│   │   └── ticket.repository.js
│   ├── validators/          # kiểm tra dữ liệu vào
│   │   └── ticket.validator.js
│   ├── errors/              # các lớp lỗi của riêng ứng dụng
│   │   └── AppError.js
│   └── app.js               # ghép các tầng lại, không có logic
├── migrations/              # các bước tạo và sửa bảng, chạy theo thứ tự
│   ├── 001_create_customers.sql
│   └── 002_create_tickets.sql
├── seeds/                   # dữ liệu mẫu để hệ thống có gì mà xem
│   └── 001_seed_demo.sql
├── tests/
│   ├── ticket.service.test.js
│   └── ticket.repository.test.js
├── .env.example             # liệt kê ĐỦ mọi biến, giá trị là giá trị mẫu
├── package.json
├── package-lock.json        # BẮT BUỘC commit - đây là tệp khoá phiên bản
└── README.md
```

Quy tắc phụ thuộc một chiều — kiểm tra bằng cách đọc câu lệnh import

routes ĐƯỢC import services. services ĐƯỢC import repositories. Không bao giờ ngược lại.

services TUYỆT ĐỐI KHÔNG được import bất cứ gì của HTTP (không có req, không có res, không có mã trạng thái HTTP trong services).

repositories TUYỆT ĐỐI KHÔNG được chứa quy tắc nghiệp vụ. Nó chỉ đọc và ghi.

Phép thử 30 giây: mở tệp service của em, tìm chữ "req", "res", "status(" và "SELECT". Nếu tìm thấy bất cứ chữ nào, tầng đã bị lẫn và phải sửa trước khi tới buổi 8.

A2. Migration — tạo bảng đúng theo DDL đã nộp ở Bài tập 1

Migration là tệp SQL (hoặc tệp mã của công cụ migration) chạy theo thứ tự để dựng cấu trúc cơ sở dữ liệu từ đầu. Ba yêu cầu của Bài tập 2 với migration: chạy từ CSDL trống phải dựng đủ bảng · chạy lần hai không lỗi · khớp đúng DDL đã nộp ở BT1, hoặc nếu khác thì phải ghi lý do vào docs/thay-doi-thiet-ke.md.

migrations/002_create_tickets.sql — chú ý IF NOT EXISTS và INDEX

```
-- Bảng phiếu bảo hành, tương ứng luồng nghiệp vụ L2
CREATE TABLE IF NOT EXISTS tickets (
  ticket_id      BIGSERIAL PRIMARY KEY,
  ticket_code    VARCHAR(20)  NOT NULL UNIQUE,
  customer_id    BIGINT       NOT NULL REFERENCES customers(customer_id),
  serial_no      VARCHAR(40)  NOT NULL,
  product_id     BIGINT       NOT NULL REFERENCES products(product_id),
  center_id      INT          NOT NULL REFERENCES service_centers(center_id),
  category_id    INT          NOT NULL REFERENCES issue_categories(category_id),
  priority       VARCHAR(12)  NOT NULL,
  status         VARCHAR(20)  NOT NULL DEFAULT 'MOI_TIEP_NHAN',
  technician_id  BIGINT       NULL REFERENCES technicians(technician_id),
```

```
  is_warranty    BOOLEAN      NOT NULL,
  received_at    TIMESTAMP    NOT NULL DEFAULT NOW(),
  due_date       TIMESTAMP    NOT NULL,
  closed_at      TIMESTAMP    NULL,
  CONSTRAINT chk_priority CHECK (priority IN ('CAO','TRUNG_BINH','THAP')),
  CONSTRAINT chk_closed   CHECK (closed_at IS NULL OR closed_at >= received_at)
);

-- INDEX cho các cột dùng để LỌC và SẮP XẾP.
-- Đây là chỗ yêu cầu phi chức năng về hiệu năng biến thành thiết kế thật.
CREATE INDEX IF NOT EXISTS idx_tickets_serial   ON tickets(serial_no);
CREATE INDEX IF NOT EXISTS idx_tickets_status   ON tickets(status);
CREATE INDEX IF NOT EXISTS idx_tickets_center_r ON tickets(center_id, received_at DESC);
```

Ba lỗi migration khiến giảng viên không dựng lại được hệ thống

Thiếu IF NOT EXISTS. Chạy lần hai báo lỗi "relation already exists" và cả script dừng. Đây là lỗi vi phạm nguyên tắc TÁI LẬP ĐƯỢC.

Migration không có thứ tự rõ ràng. Đặt tên có số ở đầu (001_, 002_) và ghi trong README thứ tự chạy, hoặc dùng công cụ migration của framework.

Chỉ có bảng, không có dữ liệu mẫu. Hệ thống dựng lên trống trơn, người chấm không thấy gì để đánh giá. Phải có tệp seed nạp ít nhất 20–30 bản ghi mỗi bảng chính.

A3. Ba tầng của một nghiệp vụ — mã mẫu hoàn chỉnh trên luồng L2

Dưới đây là toàn bộ ba tầng cho một nghiệp vụ thật: tiếp nhận một thiết bị vào bảo hành. Đọc kỹ ranh giới trách nhiệm của từng tầng — đó là điều tài liệu này muốn dạy, không phải cú pháp của thư viện.

Tầng 3 — Repository: chỉ đọc và ghi, không quyết định gì

src/repositories/ticket.repository.js

```
// Tầng này CHỈ biết đến CSDL. Không có quy tắc nghiệp vụ nào ở đây.
class TicketRepository {
  constructor(db) { this.db = db; }

  async findBySerial(serialNo) {
    const { rows } = await this.db.query(
      `SELECT t.*, p.warranty_months, p.product_name
         FROM tickets t JOIN products p ON p.product_id = t.product_id
        WHERE t.serial_no = $1
        ORDER BY t.received_at DESC`, [serialNo]);
    return rows;
  }

  async findOpenByTechnician(technicianId) {
    const { rows } = await this.db.query(
      `SELECT ticket_id FROM tickets
        WHERE technician_id = $1 AND status NOT IN ('HOAN_TAT','HUY')`,
      [technicianId]);
    return rows;
  }

  async insert(data) {
    const { rows } = await this.db.query(
      `INSERT INTO tickets (ticket_code, customer_id, serial_no, product_id,
          center_id, category_id, priority, is_warranty, due_date)
       VALUES ($1,$2,$3,$4,$5,$6,$7,$8,$9) RETURNING *`,
      [data.ticketCode, data.customerId, data.serialNo, data.productId,
       data.centerId, data.categoryId, data.priority, data.isWarranty, data.dueDate]);
    return rows[0];
  }
  async updateTechnician(ticketId, technicianId) {
    const { rows } = await this.db.query(
      'UPDATE tickets SET technician_id = $2 WHERE ticket_id = $1 RETURNING *',
      [ticketId, technicianId]);
```

```
    return rows[0];
  }
  // Sinh mã phiếu PB-<mã trung tâm>-<số thứ tự 5 chữ số>, ví dụ PB-2-00007.
  // Đủ cho prototype; hệ thống nhiều người dùng đồng thời nên dùng SEQUENCE của CSDL.
  async nextTicketCode(centerId) {
    const { rows } = await this.db.query(
      'SELECT COUNT(*)::int AS n FROM tickets WHERE center_id = $1', [centerId]);
    return `PB-${centerId}-${String(rows[0].n + 1).padStart(5, '0')}`;
  }
}
module.exports = TicketRepository;
```

Tầng 2 — Service: toàn bộ quy tắc nghiệp vụ, không biết gì về HTTP

src/services/ticket.service.js — chú ý: không có req, không có res, không có số 400/409

```
const { AppError } = require('../errors/AppError');

// SLA theo mức độ ưu tiên, đơn vị giờ - quy tắc nghiệp vụ QT-08
const SLA_HOURS = { CAO: 24, TRUNG_BINH: 72, THAP: 120 };
const MAX_OPEN_PER_TECH = 8;             // quy tắc nghiệp vụ QT-11

class TicketService {
  constructor(ticketRepo, customerRepo, techRepo) {
    this.tickets = ticketRepo;
    this.customers = customerRepo;
    this.techs = techRepo;
  }

  // QT-05: thiết bị còn hạn bảo hành khi chưa quá warranty_months kể từ ngày bán
  isUnderWarranty(soldAt, warrantyMonths, now = new Date()) {
    const expiry = new Date(soldAt);
    expiry.setMonth(expiry.getMonth() + warrantyMonths);
    return now <= expiry;
  }

  async receiveDevice(input) {
    // 1. Kiểm tra tồn tại - quy tắc nghiệp vụ, không phải kiểm tra định dạng
    const customer = await this.customers.findById(input.customerId);
    if (!customer) throw new AppError('KHACH_KHONG_TON_TAI',
      'Không tìm thấy khách hàng với mã đã cho.');

    // 2. QT-09: một serial không được có hai phiếu đang mở cùng lúc
    const existing = await this.tickets.findBySerial(input.serialNo);
    const openOne = existing.find(t => !['HOAN_TAT','HUY'].includes(t.status));
    if (openOne) throw new AppError('PHIEU_DANG_MO',
      `Thiết bị ${input.serialNo} đã có phiếu ${openOne.ticket_code} đang xử lý.`);

    // 3. Tính trạng thái bảo hành
    const isWarranty = this.isUnderWarranty(
      input.soldAt, input.warrantyMonths);

    // 4. Tính hạn xử lý từ SLA
    const hours = SLA_HOURS[input.priority];
    if (!hours) throw new AppError('UU_TIEN_KHONG_HOP_LE',
      'Mức độ ưu tiên phải là CAO, TRUNG_BINH hoặc THAP.');
    const dueDate = new Date(Date.now() + hours * 3600 * 1000);

    // 5. Tạo phiếu
    return this.tickets.insert({ ...input, isWarranty, dueDate,
      ticketCode: await this.tickets.nextTicketCode(input.centerId) });
  }

  // QT-11: không phân công quá MAX_OPEN_PER_TECH phiếu đang mở cho một kỹ thuật viên
  async assignTechnician(ticketId, technicianId) {
    const tech = await this.techs.findById(technicianId);
    if (!tech || !tech.is_active) throw new AppError('KTV_KHONG_KHA_DUNG',
      'Kỹ thuật viên không tồn tại hoặc đã ngừng hoạt động.');
    const open = await this.tickets.findOpenByTechnician(technicianId);
    if (open.length >= MAX_OPEN_PER_TECH) throw new AppError('KTV_QUA_TAI',
      `Kỹ thuật viên đang có ${open.length} phiếu mở, vượt giới hạn ${MAX_OPEN_PER_TECH}.`);
    return this.tickets.updateTechnician(ticketId, technicianId);
  }
```

```
}
module.exports = TicketService;
```

Vì sao service không được biết mã trạng thái HTTP

Cùng một quy tắc nghiệp vụ có thể được gọi từ REST API, từ một lệnh dòng lệnh, từ một tác vụ định kỳ, hoặc từ một bài test. Chỉ REST API mới có mã trạng thái HTTP.

Service ném ra một lỗi có MÃ NGHIỆP VỤ (PHIEU_DANG_MO, KTV_QUA_TAI). Tầng routes mới dịch mã nghiệp vụ đó thành mã HTTP tương ứng.

Nhờ vậy em test được toàn bộ quy tắc nghiệp vụ mà không cần khởi động máy chủ web — đây là điều làm test nhanh và ổn định.

Đây cũng chính là câu trả lời cho câu hỏi chuẩn số 1 ("vì sao em chọn cách này") khi giảng viên hỏi về kiến trúc của em ở buổi 8.

Tầng 1 — Routes: dịch giữa HTTP và nghiệp vụ, không có logic

src/routes/ticket.routes.js

```
const express = require('express');
const { validateReceive } = require('../validators/ticket.validator');

// Bản đồ dịch MÃ NGHIỆP VỤ -> MÃ HTTP. Toàn bộ phần "biết về HTTP" nằm ở đây.
const HTTP_BY_CODE = {
  DU_LIEU_KHONG_HOP_LE:  400,   // tầng 1: sai định dạng, kèm danh sách trường
  KHACH_KHONG_TON_TAI:   404,
  PHIEU_DANG_MO:         409,   // xung đột trạng thái
  UU_TIEN_KHONG_HOP_LE:  400,
  KTV_KHONG_KHA_DUNG:    404,
  KTV_QUA_TAI:           422,   // hợp lệ về cú pháp, không xử lý được về nghiệp vụ
};

module.exports = function ticketRoutes(ticketService) {
  const r = express.Router();

  r.post('/tickets', async (req, res, next) => {
    try {
      const input = validateReceive(req.body);        // tầng kiểm tra định dạng
      const ticket = await ticketService.receiveDevice(input);
      res.status(201).json({ data: ticket });
    } catch (e) {
      if (e.code && HTTP_BY_CODE[e.code])
        return res.status(HTTP_BY_CODE[e.code]).json({ error: e.code, message: e.message, chi_tiet:
e.chiTiet });
      next(e);                                        // lỗi không lường trước -> 500
    }
  });

  return r;
};
```

Hai tệp hỗ trợ để mã mẫu chạy được: AppError và validator

Ba tầng ở trên dùng hai tệp chưa được trình bày: lớp lỗi AppError và hàm validateReceive. Bản rút gọn dưới đây đủ để ba tầng chạy được; repo mẫu mekong-se-baohanh trong thư mục Học liệu có bản đầy đủ hơn.

src/errors/AppError.js

```
// src/errors/AppError.js — lỗi nghiệp vụ có MÃ, không mang mã HTTP
class AppError extends Error {
  constructor(code, message, chiTiet = undefined) {
    super(message);
    this.name = 'AppError';
    this.code = code;              // ví dụ 'PHIEU_DANG_MO' — routes dịch sang HTTP
    if (chiTiet !== undefined) this.chiTiet = chiTiet;   // ví dụ danh sách trường sai
```

```
  }
}
module.exports = { AppError };
```

src/validators/ticket.validator.js

```
// src/validators/ticket.validator.js — TẦNG 1: định dạng và kiểu, CHƯA tra CSDL
const { AppError } = require('../errors/AppError');
const UU_TIEN = ['CAO', 'TRUNG_BINH', 'THAP'];
const RE_SERIAL = /^[A-Za-z0-9-]{6,40}$/;

function validateReceive(body = {}) {
  const loi = [];                                  // gom TẤT CẢ lỗi rồi báo một lần
  for (const f of ['customerId', 'productId', 'centerId', 'categoryId'])
    if (!Number.isInteger(body[f]) || body[f] <= 0)
      loi.push({ truong: f, ly_do: 'phải là số nguyên dương' });
  const serial = typeof body.serialNo === 'string' ? body.serialNo.trim() : '';
  if (!RE_SERIAL.test(serial))
    loi.push({ truong: 'serialNo', ly_do: 'chữ, số, gạch ngang; dài 6–40 ký tự' });
  if (!UU_TIEN.includes(body.priority))
    loi.push({ truong: 'priority', ly_do: 'phải là CAO, TRUNG_BINH hoặc THAP' });
  if (!/^\d{4}-\d{2}-\d{2}$/.test(body.soldAt || '') || Number.isNaN(Date.parse(body.soldAt)))
    loi.push({ truong: 'soldAt', ly_do: 'phải theo định dạng YYYY-MM-DD' });
  if (!Number.isInteger(body.warrantyMonths) || body.warrantyMonths < 0)
    loi.push({ truong: 'warrantyMonths', ly_do: 'phải là số nguyên không âm' });
  if (loi.length) throw new AppError('DU_LIEU_KHONG_HOP_LE',
    `Dữ liệu vào không hợp lệ ở các trường: ${loi.map(l => l.truong).join(', ')}`, loi);
  return { ...body, serialNo: serial };            // trả về dữ liệu ĐÃ chuẩn hoá
}
module.exports = { validateReceive };
```

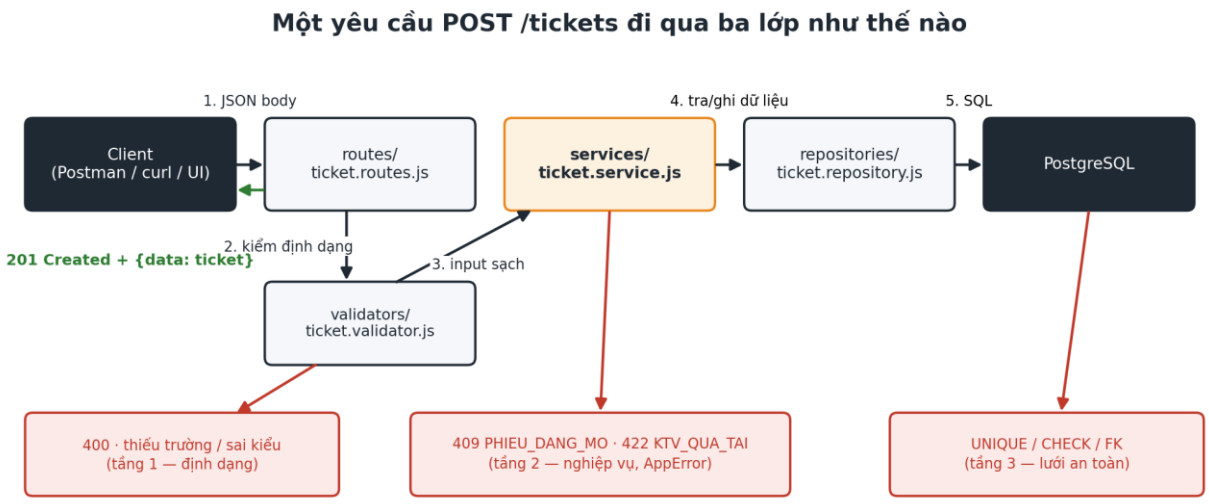
Ba điểm cần nhận ra trong validator

Gom hết lỗi rồi báo một lần. Người dùng sửa một lần là xong, không phải gửi đi gửi lại từng trường.

Không tra cơ sở dữ liệu. "Khách hàng có tồn tại không" là câu hỏi nghiệp vụ, thuộc tầng 2 (service).

Trả về dữ liệu đã chuẩn hoá. Service nhận serial đã được cắt khoảng trắng, nên không phải kiểm lại.

Minh hoạ: đường đi của một yêu cầu và nơi mỗi loại lỗi bị chặn



Hình 2. Lỗi định dạng dừng ở validator (400), lỗi nghiệp vụ dừng ở service (409/422), ràng buộc CSDL là lưới an toàn cuối cùng. Chỉ routes biết mã HTTP.

Ví dụ: gọi thử bằng curl và đọc phản hồi

Ba lần gọi — ba tầng phản ứng khác nhau

```
# (1) Luồng đúng -> 201 Created
curl -i -X POST http://localhost:3000/tickets -H 'Content-Type: application/json' \
  -d '{"customerId":1,"serialNo":"SN-10293","productId":5,"centerId":2,
       "categoryId":1,"priority":"CAO","soldAt":"2026-03-01","warrantyMonths":12}'
HTTP/1.1 201 Created
{"data":{"ticket_id":101,"ticket_code":"PB-2-00007","status":"MOI_TIEP_NHAN",
         "is_warranty":true,"due_date":"2026-10-13T09:15:00.000Z", ...}}

# (2) Thiếu priority, serial quá ngắn -> 400, tầng 1 chặn, CHƯA chạm CSDL
HTTP/1.1 400 Bad Request
{"error":"DU_LIEU_KHONG_HOP_LE",
 "message":"Dữ liệu vào không hợp lệ ở các trường: serialNo, priority",
 "chi_tiet":[{"truong":"serialNo","ly_do":"chữ, số, gạch ngang; dài 6–40 ký tự"},
             {"truong":"priority","ly_do":"phải là CAO, TRUNG_BINH hoặc THAP"}]}

# (3) Gửi lại đúng yêu cầu (1) -> 409, tầng 2 chặn theo QT-09
HTTP/1.1 409 Conflict
{"error":"PHIEU_DANG_MO","message":"Thiết bị SN-10293 đã có phiếu PB-2-00007 đang xử lý."}
```

Đối chiếu với các ảnh chụp minh chứng phải nộp

Lần gọi (1) là minh chứng luồng đúng; (2) là minh chứng tầng 1 nêu rõ TÊN trường sai; (3) là minh chứng một quy tắc nghiệp vụ thật (QT-09). Ba ảnh chụp này đưa thẳng vào mục 3 của file PDF Bài tập 2.

Phản hồi lỗi không bao giờ chứa vết ngăn xếp (stack trace), tên bảng hay câu SQL. Nếu thấy những thứ đó, routes đang để lọt lỗi thô ra ngoài.

A4. Ba tầng kiểm tra dữ liệu vào — mỗi tầng bắt một loại lỗi khác

Tầng	Kiểm cái gì	Ở đâu trong code	Ví dụ cụ thể
1. Định dạng và kiểu	Trường bắt buộc có mặt · kiểu đúng · độ dài hợp lệ · giá trị thuộc tập cho phép	validators/, chạy trước khi vào service	serial_no là chuỗi 6–40 ký tự (chữ, số, gạch ngang) · priority thuộc {CAO, TRUNG_BINH, THAP} · customer_id là số nguyên dương
2. Quy tắc nghiệp vụ	Các điều kiện chỉ biết được khi tra dữ liệu hiện có	services/, ném AppError có mã nghiệp vụ	Serial này chưa có phiếu nào đang mở · kỹ thuật viên chưa quá tải · khách hàng tồn tại
3. Ràng buộc cơ sở dữ liệu	Lưới an toàn cuối cùng, chặn cả trường hợp code có lỗi	migrations/, bằng UNIQUE, CHECK, FOREIGN KEY, NOT NULL	ticket_code UNIQUE · priority CHECK · closed_at >= received_at

Vì sao cần cả ba tầng mà không thể chỉ dùng một

Chỉ có tầng 1: hệ thống nhận dữ liệu đúng định dạng nhưng sai nghiệp vụ — tạo được hai phiếu mở cho cùng một thiết bị.

Chỉ có tầng 2: mọi yêu cầu sai định dạng đều phải chạy qua truy vấn cơ sở dữ liệu mới bị từ chối, tốn tài nguyên và thông báo lỗi kém rõ ràng.

Chỉ có tầng 3: người dùng nhận được thông báo lỗi của cơ sở dữ liệu — vừa khó hiểu vừa để lộ cấu trúc bên trong hệ thống, là một vấn đề bảo mật.

Tầng	Kiểm cái gì	Ở đâu trong code	Ví dụ cụ thể

Câu hỏi thầy sẽ hỏi ở buổi 8: "Nếu thầy gửi một yêu cầu tạo phiếu với priority là chuỗi rỗng, thì tầng nào của em chặn nó, và người dùng nhận được thông báo gì?" Hãy chuẩn bị trả lời bằng cách chỉ vào code.

A5. Unit test cho lớp service — dùng repository giả

Test lớp service là chỗ đáng bỏ công nhất, vì đó là nơi chứa quy tắc nghiệp vụ. Vì service không biết gì về HTTP và nhận repository qua hàm khởi tạo, em thay repository thật bằng một đối tượng giả và test chạy trong vài mili giây, không cần cơ sở dữ liệu.

tests/ticket.service.test.js — ba test bắt buộc cho Bài tập 2

```
const TicketService = require('../src/services/ticket.service');

// Repository giả: chỉ trả về dữ liệu ta muốn, không kết nối CSDL nào
const fakeCustomers = { findById: async id => (id === 1 ? { customer_id: 1 } : null) };
const fakeTechs     = { findById: async id => ({ technician_id: id, is_active: true }) };

function makeTickets(openTickets = [], existingForSerial = []) {
  return {
    findBySerial:          async () => existingForSerial,
    findOpenByTechnician:  async () => openTickets,
    insert:                async d  => ({ ticket_id: 99, ...d }),
    updateTechnician:      async () => ({ ok: true }),
    nextTicketCode:        async () => 'PB-2-00007',
  };
}

test('TC-01 luồng đúng: tiếp nhận thiết bị còn hạn bảo hành thì tạo được phiếu', async () => {
  const svc = new TicketService(makeTickets(), fakeCustomers, fakeTechs);
  const t = await svc.receiveDevice({
    customerId: 1, serialNo: 'SN-10293', productId: 5, centerId: 2,
    categoryId: 1, priority: 'CAO',
    soldAt: '2026-03-01', warrantyMonths: 12,
  });
  expect(t.isWarranty).toBe(true);
  expect(t.ticketCode).toBe('PB-2-00007');
  expect(t.dueDate.getTime()).toBeGreaterThan(Date.now());
});

test('TC-02 giá trị biên: ngày cuối còn bảo hành, ngày kế tiếp thì hết', () => {
  const svc = new TicketService(makeTickets(), fakeCustomers, fakeTechs);
  // bán 01/03/2026, bảo hành 12 tháng -> hết hạn 01/03/2027
  expect(svc.isUnderWarranty('2026-03-01', 12, new Date('2027-03-01'))).toBe(true);
  expect(svc.isUnderWarranty('2026-03-01', 12, new Date('2027-03-02'))).toBe(false);
});

test('TC-03 dữ liệu sai: serial đã có phiếu đang mở thì bị từ chối', async () => {
  const svc = new TicketService(
    makeTickets([], [{ ticket_code: 'PB-2-00007', status: 'DANG_SUA' }]),
    fakeCustomers, fakeTechs);
  await expect(svc.receiveDevice({
    customerId: 1, serialNo: 'SN-10293', productId: 5, centerId: 2,
    categoryId: 1, priority: 'CAO', soldAt: '2026-03-01', warrantyMonths: 12,
  })).rejects.toMatchObject({ code: 'PHIEU_DANG_MO' });
});
```

Ba test này ứng đúng ba loại test mà Bài tập 2 yêu cầu

TC-01 — luồng đúng: chứng minh chức năng chạy được trong điều kiện bình thường.

TC-02 — giá trị biên: ngày cuối cùng còn bảo hành và ngày đầu tiên hết bảo hành. Đây là loại test bắt được nhiều lỗi nhất trên thực tế.

TC-03 — dữ liệu sai: chứng minh hệ thống TỪ CHỐI đúng cách, và từ chối với mã lỗi cụ thể chứ không phải sập.

Cách chạy: `npm test` (trong package.json: "test": "jest") — ghi đúng lệnh này vào README. Giảng viên sẽ chạy chính lệnh đó.

Ví dụ: kết quả mong đợi khi chạy npm test

Terminal

```
$ npm test
> mekong-baohanh@0.2.0 test
> jest

 PASS  tests/ticket.service.test.js
  ✓ TC-01 luồng đúng: tiếp nhận thiết bị còn hạn bảo hành thì tạo được phiếu (4 ms)
  ✓ TC-02 giá trị biên: ngày cuối còn bảo hành, ngày kế tiếp thì hết (1 ms)
  ✓ TC-03 dữ liệu sai: serial đã có phiếu đang mở thì bị từ chối (1 ms)

Test Suites: 1 passed, 1 total
Tests:       3 passed, 3 total
Time:        0.41 s
```

Màn hình này chính là minh chứng của mục kiểm chứng C4

Chụp toàn bộ màn hình, phải thấy rõ dòng "Tests: 3 passed". Tên test nên mô tả quy tắc như trên — người chấm đọc tên test là biết em đã kiểm điều gì.

A6. Checklist tự kiểm trước khi tới workshop buổi 8 [SE]

- ☐ Chạy được từ CSDL trống: chạy migration rồi chạy seed rồi khởi động ứng dụng, không lỗi.
- ☐ Chạy migration lần thứ hai không báo lỗi (đã có IF NOT EXISTS hoặc công cụ migration lo việc này).
- ☐ Có ít nhất hai entity với đầy đủ bốn thao tác tạo, đọc, sửa, xoá.
- ☐ Mở tệp service và không tìm thấy chữ "req", "res", "status(", "SELECT".
- ☐ Mở tệp repository và không tìm thấy quy tắc nghiệp vụ nào (không có if kiểm tra điều kiện nghiệp vụ).
- ☐ Có tệp validator riêng, và một yêu cầu thiếu trường bắt buộc trả về mã 400 kèm thông báo nói rõ trường nào thiếu.
- ☐ Có ít nhất ba unit test cho service, chạy PASS bằng một lệnh.
- ☐ Đã commit package-lock.json (hoặc tệp khoá phiên bản tương đương).
- ☐ Tệp .env thật KHÔNG có trong repo; có .env.example liệt kê đủ biến.
- ☐ README ghi đúng các lệnh: cài đặt, chạy migration, chạy seed, khởi động, chạy test.

PHẦN B — Track DA: Pipeline ETL có kiểm chất lượng và chạy lại được

B1. Cấu trúc thư mục bắt buộc

Cấu trúc thư mục cho một pipeline ETL

```
du-an/
├── src/
│   ├── extract/            # đọc dữ liệu thô, KHÔNG biến đổi gì
│   │   └── read_sources.py
│   ├── transform/          # làm sạch và biến đổi, KHÔNG đọc ghi CSDL
│   │   ├── clean_customers.py
│   │   └── rules.py        # các quy tắc chuẩn hoá dùng chung
│   ├── quality/            # kiểm chất lượng, mỗi quy tắc một hàm
│   │   └── checks.py
│   ├── load/               # ghi vào đích, KHÔNG biến đổi gì
│   │   └── load_staging.py
│   └── pipeline.py         # điều phối: extract -> transform -> quality -> load
├── sql/
│   ├── 001_create_staging.sql
│   └── 002_create_reject.sql
├── config/
│   └── config.example.yaml  # đường dẫn nguồn, ngưỡng chất lượng
├── tests/
│   ├── test_rules.py
│   └── test_checks.py
├── data/                   # trong repo chỉ để tệp .gitkeep, KHÔNG commit dữ liệu
├── reports/                # báo cáo chất lượng sinh ra sau mỗi lần chạy
├── requirements.txt        # ghi phiên bản CHÍNH XÁC, không dùng >=
└── README.md
```

Ranh giới ba pha — vi phạm là mất điểm ở tiêu chí Chất lượng kỹ thuật

extract KHÔNG được biến đổi dữ liệu. Nó chỉ đọc và trả về dữ liệu thô. Nếu extract vừa đọc vừa sửa định dạng ngày thì em không còn biết dữ liệu gốc trông như thế nào.

transform KHÔNG được đọc ghi cơ sở dữ liệu. Nó nhận dữ liệu vào và trả dữ liệu ra. Nhờ vậy em test được nó mà không cần cơ sở dữ liệu — giống như tầng service của track SE.

load KHÔNG được biến đổi dữ liệu. Nó chỉ ghi. Nếu load còn sửa dữ liệu thì báo cáo chất lượng của em không phản ánh đúng thứ được ghi vào.

Phép thử: trong tệp transform, tìm chữ "connect", "cursor", "to_sql", "read_sql". Trong tệp load, tìm chữ "replace", "strip", "astype". Nếu tìm thấy, ranh giới đã bị vi phạm.

B2. Chuẩn hoá dữ liệu thật của bộ dữ liệu Mekong Mobile

Bộ dữ liệu đã phát có các vấn đề chất lượng được cài chủ ý. Bảng dưới liệt kê chính xác những gì em sẽ gặp và ở tệp nào — em không phải mất thời gian tự khám phá.

Vấn đề	Tệp và cột	Biểu hiện cụ thể	Cách xử lý đề xuất
Bốn định dạng số điện thoại	customers_raw.csv · so_dien_thoai	0763247885 · 0760.833.592 · 076 083 3592 · +84760833592	Bỏ mọi ký tự không phải chữ số, chuyển tiền tố +84 thành 0, kiểm tra còn đúng 10 chữ số.
Ba định dạng ngày	orders_2024_2026.csv · ngay	17/08/2025 (62%) · 17-8-25 (24%, năm hai chữ số, không đệm số 0) · 2025.08.17 (14%) · 0,44% ô trống	Thử lần lượt ba khuôn dạng %d/%m/%Y · %d-%m-%y · %Y.%m.%d theo thứ tự phổ biến; dòng không khớp khuôn nào thì đưa vào bảng reject. Chú ý khuôn thứ hai: năm hai

Vấn đề	Tệp và cột	Biểu hiện cụ thể	Cách xử lý đề xuất
			chữ số và không đệm số 0 — đây là khuôn dễ đọc sai nhất.
Thiếu số điện thoại	customers_raw.csv · so_dien_thoai	khoảng 6,1% dòng để trống	KHÔNG loại bỏ dòng. Đánh dấu và đếm — số điện thoại trống vẫn là một khách hàng thật.
Khách hàng trùng lặp	customers_raw.csv	khoảng 20% bản ghi là trùng của bản ghi khác, khác nhau ở định dạng tên và số điện thoại	Hợp nhất theo khoá tổng hợp: số điện thoại đã chuẩn hoá + tên đã chuẩn hoá (bỏ dấu, chữ thường, gộp khoảng trắng).
Tiền không khớp	order_items.csv	khoảng 1,5% dòng có thanh_tien ≠ so_luong × don_gia	Đưa vào bảng reject kèm cả hai giá trị. TUYỆT ĐỐI KHÔNG tự sửa lại số tiền — dữ liệu tiền phải do người có thẩm quyền quyết định.
Mã đơn trùng giữa cửa hàng	orders_2024_2026.csv · ma_don	khoảng 1.100 mã đơn xuất hiện ở nhiều cửa hàng khác nhau	Khoá thật là cặp (ma_cua_hang, ma_don), không phải ma_don một mình. Ghi phát hiện này vào Data dictionary.

B3. Mã mẫu — chuẩn hoá và kiểm chất lượng

src/transform/rules.py — mỗi quy tắc là một hàm thuần, test được ngay

```
import re
import unicodedata
from datetime import datetime

DATE_FORMATS = ['%d/%m/%Y', '%d-%m-%y', '%Y.%m.%d']   # theo thứ tự phổ biến

def chuan_hoa_dien_thoai(raw):
    """Trả về số 10 chữ số bắt đầu bằng 0, hoặc None nếu không hợp lệ."""
    if raw is None or str(raw).strip() == '':
        return None
    digits = re.sub(r'\D', '', str(raw))
    if digits.startswith('84') and len(digits) == 11:
        digits = '0' + digits[2:]
    return digits if len(digits) == 10 and digits.startswith('0') else None

def chuan_hoa_ten(raw):
    """Bỏ dấu, về chữ thường, gộp khoảng trắng - dùng LÀM KHOÁ hợp nhất."""
    if raw is None:
        return ''
    s = unicodedata.normalize('NFD', str(raw))
    s = ''.join(c for c in s if unicodedata.category(c) != 'Mn')
    return re.sub(r'\s+', ' ', s).strip().lower()

def doc_ngay(raw):
    """Thử lần lượt ba khuôn dạng. Trả về None nếu không khớp khuôn nào."""
    s = str(raw).strip()
    for fmt in DATE_FORMATS:
        try:
            return datetime.strptime(s, fmt).date()
        except ValueError:
            continue
    return None

def khoa_hop_nhat(ho_ten, so_dien_thoai):
    """Khoá tổng hợp để gom các bản ghi của cùng một khách hàng."""
    return (chuan_hoa_dien_thoai(so_dien_thoai) or 'KHONG_SDT',
            chuan_hoa_ten(ho_ten))
```

src/quality/checks.py — mỗi quy tắc trả về MỘT SỐ ĐO ĐƯỢC

```
from dataclasses import dataclass

@dataclass
class KetQuaKiem:
    ma: str             # CL1..CL5
    ten: str
    so_dong_kiem: int
    so_dong_loi: int
    nguong_cho_phep: float   # tỉ lệ lỗi tối đa cho phép
    @property
    def ti_le_loi(self):
        return self.so_dong_loi / self.so_dong_kiem if self.so_dong_kiem else 0.0
    @property
    def dat(self):
        return self.ti_le_loi <= self.nguong_cho_phep

def cl1_dien_thoai_hop_le(df, nguong=0.10):
    loi = df['sdt_chuan'].isna() & df['so_dien_thoai'].notna() \
          & (df['so_dien_thoai'].astype(str).str.strip() != '')
    return KetQuaKiem('CL1', 'Số điện thoại chuẩn hoá được',
                      len(df), int(loi.sum()), nguong)

def cl2_ngay_hop_le(df, nguong=0.01):
    loi = df['ngay_chuan'].isna()
    return KetQuaKiem('CL2', 'Ngày đọc được theo một trong ba khuôn',
                      len(df), int(loi.sum()), nguong)

def cl3_tien_khop(df, nguong=0.02):
    loi = (df['thanh_tien'] - df['so_luong'] * df['don_gia']).abs() > 1
    return KetQuaKiem('CL3', 'thanh_tien = so_luong x don_gia',
                      len(df), int(loi.sum()), nguong)

def cl4_khoa_khong_trung(df, nguong=0.0):
    loi = df.duplicated(subset=['ma_cua_hang', 'ma_don'], keep=False)
    return KetQuaKiem('CL4', 'Cặp (ma_cua_hang, ma_don) là duy nhất',
                      len(df), int(loi.sum()), nguong)

def cl5_khoa_ngoai_ton_tai(df_items, df_orders, nguong=0.0):
    hop_le = df_items['order_id'].isin(df_orders['order_id'])
    return KetQuaKiem('CL5', 'Mỗi dòng hàng trỏ về một đơn hàng có thật',
                      len(df_items), int((~hop_le).sum()), nguong)
```

Vì sao mỗi hàm kiểm chất lượng phải trả về một SỐ, không phải True/False

Một số cho em biết mức độ: 3 dòng sai trên 26.000 dòng khác hoàn toàn 8.000 dòng sai trên 26.000 dòng, dù cả hai đều là "không đạt".

Có số thì em đặt được ngưỡng chấp nhận, và ngưỡng chính là yêu cầu phi chức năng đo được của Bài tập 1 biến thành code.

Có số thì em so sánh được giữa các lần chạy, và phát hiện được khi chất lượng dữ liệu nguồn xấu đi.

Đây chính là nguyên tắc KIỂM CHỨNG ĐƯỢC áp dụng vào track DA. Báo cáo chất lượng sinh ra sau mỗi lần chạy là minh chứng khi chấm Bài tập 2.

Ví dụ: ba loại test cho rules.py

tests/test_rules.py

```
# tests/test_rules.py — chạy bằng: pytest -q
from datetime import date
import pytest
from src.transform import rules

# LOẠI 1 — luồng đúng: đủ bốn định dạng số điện thoại CÓ THẬT trong dữ liệu
@pytest.mark.parametrize('tho, mong_doi', [
    ('0763247885',    '0763247885'),
    ('0760.833.592', '0760833592'),
    ('076 083 3592', '0760833592'),
    ('+84760833592', '0760833592'),
```

```
])
def test_TR01_chuan_hoa_dien_thoai(tho, mong_doi):
    assert rules.chuan_hoa_dien_thoai(tho) == mong_doi


# LOẠI 1 – ba khuôn dạng ngày, kể cả khuôn năm hai chữ số không đệm 0
@pytest.mark.parametrize('tho', ['17/08/2025', '17-8-25', '2025.08.17'])
def test_TR02_doc_ba_khuon_ngay(tho):
    assert rules.doc_ngay(tho) == date(2025, 8, 17)


# LOẠI 2 – giá trị biên: 9 và 11 chữ số đều KHÔNG hợp lệ, chỉ 10 chữ số mới đạt
@pytest.mark.parametrize('tho', ['076083359', '07608335921'])
def test_TR03_bien_do_dai(tho):
    assert rules.chuan_hoa_dien_thoai(tho) is None


# LOẠI 3 – dữ liệu sai: rỗng hoặc rác thì trả None, KHÔNG ném ngoại lệ
@pytest.mark.parametrize('tho', [None, '', '   ', 'abc'])
def test_TR04_du_lieu_rac(tho):
    assert rules.chuan_hoa_dien_thoai(tho) is None
    assert rules.doc_ngay(tho) is None
```

Vì sao dùng parametrize

Một hàm test, nhiều bộ dữ liệu: mỗi bộ được đếm là một test riêng khi chạy pytest. Dữ liệu thử lấy đúng từ các định dạng đã liệt kê ở bảng B2 — test phản ánh dữ liệu thật, không phải dữ liệu em tự nghĩ ra.

Ví dụ: báo cáo chất lượng sinh ra sau một lần chạy

Mã	Quy tắc	Dòng kiểm	Dòng lỗi · tỉ lệ	Ngưỡng · kết luận
CL1	Số điện thoại chuẩn hoá được	53.812	0 · 0,00%	10% · ĐẠT
CL2	Ngày đọc được theo một trong ba khuôn	26.140	115 · 0,44%	1% · ĐẠT
CL3	thanh_tien = so_luong × don_gia	71.905	1.079 · 1,50%	2% · ĐẠT
CL4	Cặp (ma_cua_hang, ma_don) là duy nhất	26.140	0 · 0,00%	0% · ĐẠT
CL5	Dòng hàng trỏ về đơn hàng có thật	71.905	37 · 0,05%	0% · KHÔNG ĐẠT

Bảng minh hoạ định dạng; con số thật của em phụ thuộc dữ liệu và quy tắc làm sạch. Chú ý dòng CL5: một quy tắc KHÔNG ĐẠT vẫn phải được báo cáo trung thực, kèm số dòng đã chuyển vào bảng reject.

B4. Bảo đảm chạy lại được — hai kỹ thuật, chọn một

Kỹ thuật	Cách làm	Khi nào dùng	Nhược điểm
Xoá rồi nạp lại theo lô	Mỗi lần chạy gán một mã lô (batch_id). Trước khi nạp, xoá mọi dòng của lô cùng ngày rồi nạp mới.	Dữ liệu theo ngày hoặc theo kỳ; khối lượng vừa phải.	Tốn thời gian hơn nếu dữ liệu lớn; phải cẩn thận với điều kiện xoá.
Ghi có điều kiện theo khoá	Dùng câu lệnh chèn có xử lý trùng khoá (INSERT ... ON CONFLICT DO UPDATE / MERGE) trên khoá nghiệp vụ.	Có khoá nghiệp vụ rõ ràng và ổn định.	Phải xác định đúng khoá nghiệp vụ; sai khoá thì dữ liệu bị nhân đôi hoặc bị ghi đè oan.

Phép thử chạy lại được — làm trước khi tới workshop buổi 8

```
# Chạy lần 1
python -m src.pipeline --config config/config.yaml
psql -c "SELECT COUNT(*) FROM staging_customers;"   # ghi lại số này, ví dụ 53812
```

Kỹ thuật	Cách làm	Khi nào dùng	Nhược điểm

```
# Chạy lần 2 với ĐÚNG đầu vào đó, KHÔNG xoá gì bằng tay
python -m src.pipeline --config config/config.yaml
psql -c "SELECT COUNT(*) FROM staging_customers;"   # PHẢI ra đúng 53812

# Nếu số lần 2 = 2 x số lần 1  -> pipeline chưa chạy lại được, phải sửa
# Nếu lần 2 báo lỗi trùng khoá -> pipeline chưa chạy lại được, phải sửa
```

Ví dụ: pipeline.py — một lệnh, một lô, chạy lại không nhân đôi

src/pipeline.py

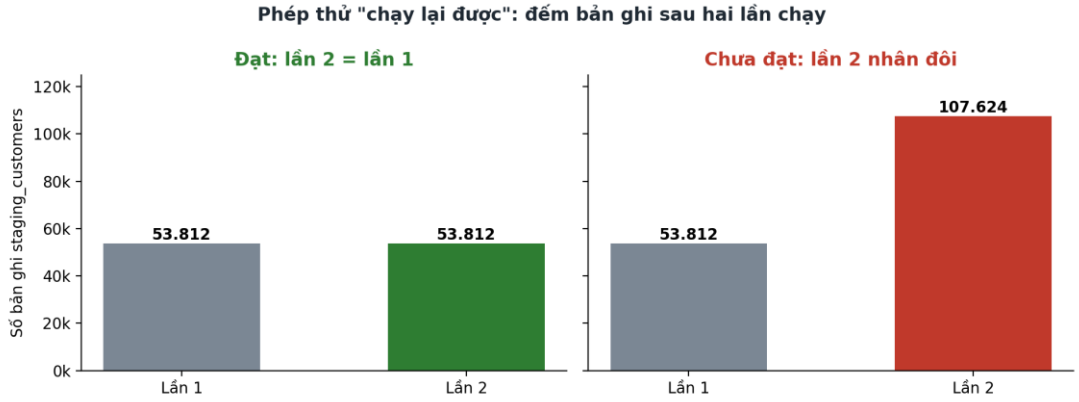
```
# src/pipeline.py — chạy bằng: python -m src.pipeline --config config/config.yaml
import argparse, datetime, yaml
from src.extract.read_sources import doc_khach_hang
from src.transform.clean_customers import lam_sach_khach_hang
from src.quality import checks
from src.load.load_staging import ghi_lo, ghi_reject, ghi_bao_cao

def main():
    ap = argparse.ArgumentParser(); ap.add_argument('--config', required=True)
    cfg = yaml.safe_load(open(ap.parse_args().config, encoding='utf-8'))
    batch_id = 'khach-hang-' + datetime.date.today().isoformat()  # một lô mỗi ngày

    tho = doc_khach_hang(cfg['nguon']['khach_hang'])          # 1. EXTRACT: chỉ đọc
    sach, reject = lam_sach_khach_hang(tho)                     # 2. TRANSFORM: hàm thuần
    ket_qua = [checks.cl1_dien_thoai_hop_le(sach)]              # 3. QUALITY: trả về SỐ

    ghi_lo(sach, batch_id)        # 4. LOAD: DELETE ... WHERE batch_id = %s rồi INSERT
    ghi_reject(reject, batch_id)  #    trong cùng một transaction -> chạy lại không nhân đôi
    ghi_bao_cao(ket_qua, batch_id, so_dong_vao=len(tho), so_reject=len(reject))
    print(f'{batch_id}: vào {len(tho)} · sạch {len(sach)} · reject {len(reject)}')

if __name__ == '__main__':
    main()
```



Hình 3. Bên trái: pipeline đạt — lần chạy thứ hai cho đúng số bản ghi như lần một. Bên phải: pipeline chỉ INSERT mà không xoá lô cũ hoặc không xử lý trùng khoá, nên số bản ghi nhân đôi.

B5. Bảng reject — nơi chứa những dòng không xử lý được

Bảng reject là thành phần bắt buộc của Bài tập 2 cho track DA. Nguyên tắc: KHÔNG bao giờ âm thầm bỏ một dòng dữ liệu. Mọi dòng không xử lý được đều phải nằm ở đâu đó, kèm lý do, để người khác truy vết được.

sql/002_create_reject.sql

```
CREATE TABLE IF NOT EXISTS reject_rows (
  reject_id    BIGSERIAL PRIMARY KEY,
  batch_id     VARCHAR(40) NOT NULL,      -- lô chạy nào
  nguon        VARCHAR(60) NOT NULL,      -- tên tệp nguồn
```

```
  so_dong      INT         NOT NULL,      -- dòng thứ mấy trong tệp nguồn
  ma_quy_tac   VARCHAR(10) NOT NULL,      -- CL1..CL5 - quy tắc nào loại nó
  ly_do        TEXT        NOT NULL,      -- viết cho NGƯỜI đọc, không phải cho máy
  du_lieu_goc  JSONB       NOT NULL,      -- toàn bộ dòng gốc, để truy vết
  tao_luc      TIMESTAMP   NOT NULL DEFAULT NOW()
);
CREATE INDEX IF NOT EXISTS idx_reject_batch ON reject_rows(batch_id, ma_quy_tac);
```

Ba yêu cầu với bảng reject khi chấm Bài tập 2

Cột lý do phải viết cho người đọc: "thanh_tien 11990000 khac so_luong 2 x don_gia 5990000 = 11980000" chứ không phải "CL3 failed".

Cột dữ liệu gốc phải giữ nguyên toàn bộ dòng nguồn, để người khác kiểm chứng lại được kết luận của pipeline.

Báo cáo chất lượng cuối mỗi lần chạy phải nêu số dòng bị reject theo từng quy tắc, và tổng số phải khớp với số dòng trong bảng reject của lô đó.

B6. Checklist tự kiểm trước khi tới workshop buổi 8 [DA]

- ☐ Pipeline chạy được bằng MỘT lệnh, ghi trong README. Không phải notebook bấm từng ô.
- ☐ Đã chạy phép thử chạy lại được: chạy lần hai cho số bản ghi giống lần một.
- ☐ Ba pha extract / transform / load nằm ở ba tệp khác nhau và không gọi chéo sai chiều.
- ☐ Có ít nhất năm hàm kiểm chất lượng CL1–CL5, mỗi hàm trả về số dòng lỗi và tỉ lệ lỗi.
- ☐ Có bảng reject và nó thực sự có dòng trong đó sau khi chạy (bộ dữ liệu chắc chắn có dòng lỗi).
- ☐ Có báo cáo chất lượng sinh ra tự động vào reports/ sau mỗi lần chạy.
- ☐ requirements.txt ghi phiên bản chính xác (dùng == chứ không dùng >=).
- ☐ Không commit tệp dữ liệu vào repo; chỉ commit .gitkeep và đường dẫn nguồn trong config.
- ☐ Có ít nhất ba test cho các hàm chuẩn hoá trong rules.py, chạy PASS bằng một lệnh.
- ☐ Đã ghi phát hiện "khoá thật là cặp (ma_cua_hang, ma_don)" vào Data dictionary.

PHẦN C — Track AI: Thu thập, làm sạch và chia dữ liệu đúng cách

C1. Cấu trúc thư mục bắt buộc

Cấu trúc thư mục cho một dự án học máy

```
du-an/
├── notebooks/
│   └── 01_eda.ipynb          # KHÁM PHÁ dữ liệu - KHÔNG phải sản phẩm nộp
├── src/
│   ├── data/
│   │   ├── load.py           # nạp dữ liệu thô
│   │   └── split.py          # CHIA dữ liệu - làm TRƯỚC mọi biến đổi
│   ├── features/
│   │   └── text.py           # làm sạch văn bản, vector hoá
│   ├── models/
│   │   ├── baseline.py       # mô hình cơ sở BẮT BUỘC để so sánh
│   │   └── train.py          # huấn luyện mô hình chính
│   └── evaluate.py           # đo và ghi kết quả
├── artifacts/                # mô hình, vectorizer đã lưu (không commit tệp lớn)
├── reports/
│   └── metrics.json          # kết quả đo, sinh tự động
├── configs/
│   └── train.yaml            # siêu tham số, hạt giống ngẫu nhiên
├── tests/
│   └── test_text.py
├── requirements.txt
└── README.md
```

Notebook không phải là sản phẩm nộp

Notebook dùng để KHÁM PHÁ: xem phân bố nhãn, xem vài mẫu dữ liệu, thử nhanh một ý tưởng. Việc đó là hợp lệ và cần thiết.

Nhưng bài toán "huấn luyện lại mô hình" phải là một script chạy bằng một lệnh, cho kết quả giống nhau mỗi lần chạy. Notebook bấm từng ô không đáp ứng được điều đó vì thứ tự chạy không xác định.

Mục kiểm chứng của Bài tập 2 cho track AI: lệnh `python -m src.models.train --config configs/train.yaml` phải chạy được và sinh ra reports/metrics.json.

Cho phép nộp kèm notebook EDA — nó là minh chứng cho việc em đã hiểu dữ liệu trước khi mô hình hoá.

C2. Việc đầu tiên phải làm: xem phân bố nhãn

Trước khi làm bất cứ điều gì khác, hãy đếm số mẫu của từng nhãn. Bước này mất 30 giây và quyết định toàn bộ những gì em làm sau đó: chọn chỉ số đánh giá nào, có cần xử lý mất cân bằng không, mô hình cơ sở nên là gì.

Bước đầu tiên trong notebook EDA

```
import pandas as pd

df = pd.read_csv('data/issue_descriptions.csv')
print(df.shape)                                    # (4000, 3)
print(df['issue_category'].value_counts())
print(df['issue_category'].value_counts(normalize=True).round(3))

# Câu hỏi phải trả lời được TRƯỚC KHI làm tiếp:
#  1. Nhãn nào nhiều nhất, chiếm bao nhiêu phần trăm?
#     -> đó là tỉ lệ chính xác của mô hình "luôn đoán nhãn đa số".
#        Nếu mô hình của em không hơn số này, nó vô dụng.
#  2. Nhãn nào ít nhất, có đủ mẫu để học không?
#     -> dưới 50 mẫu mỗi nhãn thì kết quả đo rất không ổn định.
```

```
#  3. Có nhãn nào là thùng rác không (như KHAC)?
#     -> nhãn thùng rác thường làm tụt kết quả và cần nói rõ trong báo cáo.
```

C3. Sai lầm nghiêm trọng nhất của track AI: rò rỉ dữ liệu

Quy tắc một dòng: CHIA DỮ LIỆU TRƯỚC, HỌC THAM SỐ SAU

SAI: fit vectorizer trên toàn bộ dữ liệu, rồi mới chia train/test. Vectorizer đã "nhìn thấy" từ vựng của tập test, nên kết quả đo trên tập test cao hơn thực tế và hoàn toàn vô nghĩa.

ĐÚNG: chia trước. fit vectorizer CHỈ trên tập train. Với tập test chỉ dùng transform, không fit.

Quy tắc này áp dụng cho MỌI thứ học tham số từ dữ liệu: vectorizer, bộ chuẩn hoá số, bộ điền giá trị thiếu, bộ chọn đặc trưng, bộ giảm chiều.

Vì sao đây là vấn đề đạo đức chứ không chỉ là lỗi kỹ thuật: em báo cáo một con số cao hơn năng lực thật của mô hình. Nếu ai đó tin con số đó và triển khai mô hình, hậu quả thuộc về em. Đây là nội dung câu 4 của đề thi cuối kỳ.

src/data/split.py và src/features/text.py — thứ tự đúng

```
# ---------------- src/data/split.py ----------------
from sklearn.model_selection import train_test_split

def chia_du_lieu(df, nhan='issue_category', hat_giong=42):
    """Chia 70/15/15, GIỮ NGUYÊN tỉ lệ nhãn ở cả ba tập (stratify)."""
    train, tam = train_test_split(
        df, test_size=0.30, random_state=hat_giong, stratify=df[nhan])
    val, test = train_test_split(
        tam, test_size=0.50, random_state=hat_giong, stratify=tam[nhan])
    return train, val, test


# ---------------- src/features/text.py ----------------
import re
from sklearn.feature_extraction.text import TfidfVectorizer

def lam_sach(s):
    """GIỮ DẤU tiếng Việt. Bỏ thông tin định danh. Chuẩn hoá khoảng trắng."""
    s = str(s).lower()
    s = re.sub(r'\b0\d{9}\b', ' <SDT> ', s)          # số điện thoại
    s = re.sub(r'\bSN-?\d+\b', ' <SERIAL> ', s, flags=re.I)
    s = re.sub(r'\S+@\S+', ' <EMAIL> ', s)
    s = re.sub(r'[^\w\sàáảãạăằắẳẵặâầấẩẫậèéẻẽẹêềếểễệ'
               r'ìíỉĩịòóỏõọôồốổỗộơờớởỡợùúủũụưừứửữự'
               r'ỳýỷỹỵđ<>]', ' ', s)
    return re.sub(r'\s+', ' ', s).strip()

def tao_vectorizer():
    return TfidfVectorizer(ngram_range=(1, 2), min_df=2, max_features=20000)

# ---------------- THỨ TỰ ĐÚNG trong train.py ----------------
# 1. nạp dữ liệu thô
# 2. CHIA train / val / test                    <-- làm TRƯỚC
# 3. làm sạch văn bản cho từng tập (không học tham số nên an toàn)
# 4. vec = tao_vectorizer(); X_train = vec.fit_transform(train_text)
# 5. X_val = vec.transform(val_text);  X_test = vec.transform(test_text)
#    KHÔNG BAO GIỜ gọi fit hay fit_transform trên val hoặc test
```

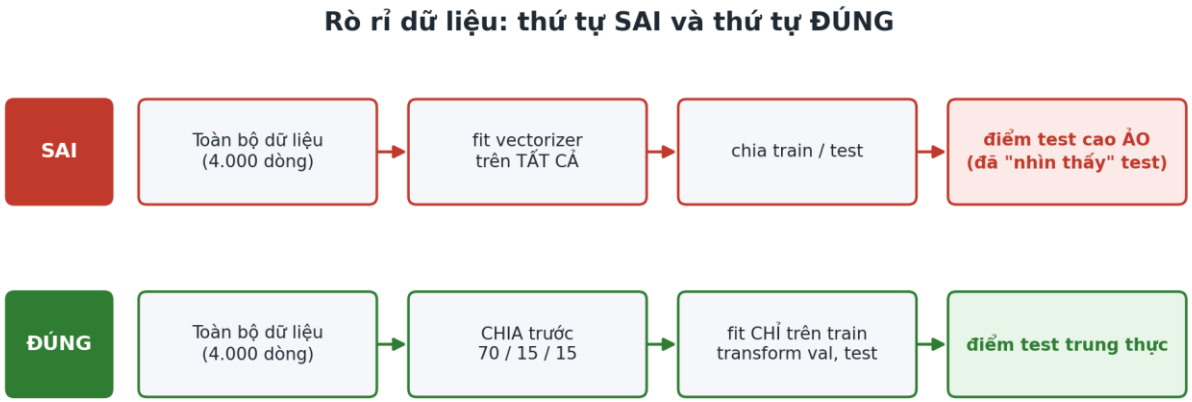
Vì sao phải GIỮ dấu tiếng Việt

Bỏ dấu làm mất nghĩa và gộp các từ khác nhau thành một: "sạc" và "sách" đều thành "sach"; "lỗi" và "lối" cũng chỉ còn là "loi".

Trong dữ liệu Mekong Mobile, nhiều mô tả sự cố chỉ khác nhau ở dấu, ví dụ "màn hình" và "man hinh" — nên chuẩn hoá chữ thường thì được, bỏ dấu thì mất thông tin.

Nếu em quyết định bỏ dấu vì lý do cụ thể (ví dụ dữ liệu nguồn có tới 40% viết không dấu), hãy ghi lý do đó vào báo cáo và ĐO cả hai phương án để chứng minh lựa chọn. Đó là truy vết được.

Minh hoạ: thứ tự sai và thứ tự đúng



Hình 4. Khác biệt duy nhất giữa hai dòng là VỊ TRÍ của bước chia dữ liệu. Thứ tự sai thường cho điểm test cao hơn vài phần trăm — chính vì thế nó nguy hiểm: con số đẹp hơn nhưng không thật.

Ví dụ: ba test cho hàm làm sạch văn bản

tests/test_text.py

```
# tests/test_text.py — chạy bằng: pytest -q
from src.features.text import lam_sach

def test_TT01_giu_dau_va_ve_chu_thuong():            # luồng đúng
    assert lam_sach('Màn Hình bị SỌC') == 'màn hình bị sọc'

def test_TT02_thay_thong_tin_dinh_danh_bang_the():    # bảo vệ dữ liệu cá nhân
    s = lam_sach('Gọi 0912345678, máy SN-10293, mail an@vd.vn')
    assert '<SDT>' in s and '<SERIAL>' in s and '<EMAIL>' in s
    assert '0912345678' not in s

def test_TT03_chuoi_rong_hoac_chi_co_khoang_trang():  # dữ liệu sai: không ném lỗi
    assert lam_sach('') == ''
    assert lam_sach('   \n\t ') == ''
```

C4. Mô hình cơ sở là bắt buộc

Mô hình cơ sở (baseline) là mô hình đơn giản nhất có thể, dùng làm mốc so sánh. Không có mô hình cơ sở thì con số của em không có ý nghĩa: 78% có thể là rất tốt hoặc rất tệ, tuỳ vào việc đoán bừa cho ra bao nhiêu.

src/models/baseline.py — hai mô hình cơ sở bắt buộc

```
from sklearn.dummy import DummyClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, f1_score

def baseline_da_so(y_train, y_test):
    """Mức sàn: luôn đoán nhãn xuất hiện nhiều nhất."""
    m = DummyClassifier(strategy='most_frequent').fit([[0]] * len(y_train), y_train)
    y_pred = m.predict([[0]] * len(y_test))
    return f1_score(y_test, y_pred, average='macro', zero_division=0)

def baseline_tfidf_logistic(X_train, y_train, X_test, y_test):
    """Mức chuẩn: TF-IDF + hồi quy logistic. Rất khó vượt qua bằng mô hình phức tạp
```

```
        trên tập dữ liệu vài nghìn dòng - hãy đo trước khi tin vào mô hình lớn."""
    m = LogisticRegression(max_iter=1000, class_weight='balanced').fit(X_train, y_train)
    y_pred = m.predict(X_test)
    print(classification_report(y_test, y_pred, zero_division=0))
    return f1_score(y_test, y_pred, average='macro', zero_division=0)
```

Ba con số phải có trong reports/metrics.json khi nộp Bài tập 2

1. F1 macro của mô hình đoán nhãn đa số. Đây là mức sàn. Mô hình của em phải cao hơn con số này, nếu không thì nó không học được gì.

2. F1 macro của TF-IDF + hồi quy logistic. Đây là mức chuẩn. Với vài nghìn dòng văn bản, mô hình này thường rất khó vượt qua.

3. F1 macro của mô hình em chọn, kèm F1 của từng nhãn. Chỉ số tổng thể che giấu việc mô hình bỏ rơi hoàn toàn một nhãn ít mẫu — phải báo cáo tách theo nhãn.

Vì sao dùng F1 macro chứ không dùng độ chính xác: độ chính xác thưởng cho việc đoán đúng nhãn đa số. F1 macro tính trung bình đều cho mọi nhãn, nên nó phát hiện được khi mô hình bỏ rơi nhãn ít mẫu. Hãy chuẩn bị trả lời câu hỏi này ở buổi 8.

C5. Bảo đảm chạy lại cho kết quả giống nhau

Bốn việc phải làm để kết quả tái lập được

```
# 1. Ghi hạt giống ngẫu nhiên vào config, KHÔNG để mặc định
#    configs/train.yaml:  hat_giong: 42
import random, numpy as np
random.seed(cfg['hat_giong']); np.random.seed(cfg['hat_giong'])

# 2. Truyền hạt giống vào MỌI chỗ có ngẫu nhiên
train_test_split(..., random_state=cfg['hat_giong'])
LogisticRegression(random_state=cfg['hat_giong'])

# 3. Ghi phiên bản thư viện CHÍNH XÁC vào requirements.txt
#    scikit-learn==1.5.2      <-- dùng ==, không dùng >=
#    pandas==2.2.3

# 4. Ghi vào metrics.json ĐỦ ĐIỀU KIỆN ĐO, không chỉ ghi con số
{
  "hat_giong": 42,
  "so_mau_train": 2800, "so_mau_val": 600, "so_mau_test": 600,
  "phien_ban_du_lieu": "issue_descriptions.csv, 4000 dòng, seed 20260901",
  "f1_macro_baseline_da_so": 0.052,
  "f1_macro_baseline_tfidf_logistic": 0.741,
  "f1_macro_mo_hinh_chinh": 0.768,
  "f1_theo_nhan": { "MAN_HINH": 0.83, "PIN": 0.79, "SAC": 0.74,
                    "PHAN_MEM": 0.71, "NUOC_VAO": 0.80, "KHAC": 0.74 }
}

# Phép thử: chạy lại train hai lần, hai tệp metrics.json phải GIỐNG NHAU.
#   python -m src.models.train --config configs/train.yaml
#   cp reports/metrics.json /tmp/lan1.json
#   python -m src.models.train --config configs/train.yaml
#   diff /tmp/lan1.json reports/metrics.json      # phải không có khác biệt
```

Các con số F1 trong ví dụ trên chỉ là minh hoạ định dạng, không phải kết quả mong đợi. Kết quả thật của em phụ thuộc vào cách em làm sạch và chọn đặc trưng.

Ví dụ: khung train.py — một lệnh từ dữ liệu thô tới metrics.json

src/models/train.py

```
# src/models/train.py — chạy bằng: python -m src.models.train --config configs/train.yaml
import argparse, json, random, yaml, numpy as np, pandas as pd
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.metrics import f1_score
from src.data.split import chia_du_lieu
from src.features.text import lam_sach, tao_vectorizer
from src.models.baseline import baseline_da_so, baseline_tfidf_logistic

def main():
    ap = argparse.ArgumentParser(); ap.add_argument('--config', required=True)
    cfg = yaml.safe_load(open(ap.parse_args().config, encoding='utf-8'))
    hg = cfg['hat_giong']; random.seed(hg); np.random.seed(hg)

    df = pd.read_csv(cfg['duong_dan_du_lieu'])                    # 1. nạp
    train, val, test = chia_du_lieu(df, hat_giong=hg)             # 2. CHIA TRƯỚC
    t_tr, t_va, t_te = (x['issue_desc'].map(lam_sach) for x in (train, val, test))
    vec = tao_vectorizer()
    X_tr = vec.fit_transform(t_tr)                                # 3. fit CHỈ trên train
    X_va, X_te = vec.transform(t_va), vec.transform(t_te)
    y_tr, y_va, y_te = train['issue_category'], val['issue_category'], test['issue_category']

    # 4. chọn C trên VALIDATION; tập test chỉ đo MỘT lần ở cuối
    def f1_val(C):
        m = LogisticRegression(C=C, max_iter=1000, class_weight='balanced', random_state=hg)
        return f1_score(y_va, m.fit(X_tr, y_tr).predict(X_va), average='macro')
    tot = max(cfg['C_thu'], key=f1_val)
    m = LogisticRegression(C=tot, max_iter=1000, class_weight='balanced',
                           random_state=hg).fit(X_tr, y_tr)
    y_pred = m.predict(X_te)
    ket_qua = {
        'hat_giong': hg, 'C_chon': tot,
        'so_mau_train': len(train), 'so_mau_val': len(val), 'so_mau_test': len(test),
        'f1_macro_baseline_da_so': round(baseline_da_so(y_tr, y_te), 3),
        'f1_macro_baseline_tfidf_logistic': round(
            baseline_tfidf_logistic(X_tr, y_tr, X_te, y_te), 3),
        'f1_macro_mo_hinh_chinh': round(f1_score(y_te, y_pred, average='macro'), 3),
        'f1_theo_nhan': {n: round(float(v), 3) for n, v in zip(m.classes_,
                         f1_score(y_te, y_pred, average=None, labels=m.classes_))},
    }
    with open('reports/metrics.json', 'w', encoding='utf-8') as f:
        json.dump(ket_qua, f, ensure_ascii=False, indent=2, sort_keys=True)  # diff ổn định

if __name__ == '__main__':
    main()
```

Đọc khung này theo bốn bước đánh số

Bước 2 đứng trước bước 3 — đó là toàn bộ nội dung mục C3 viết thành mã. Siêu tham số C được chọn trên tập validation (bước 4); tập test chỉ được dùng một lần để báo cáo. configs/train.yaml cần ba khoá: hat_giong, duong_dan_du_lieu, C_thu (ví dụ [0.3, 1, 3]).

C6. Checklist tự kiểm trước khi tới workshop buổi 8 [AI]

- ☐ Đã xem phân bố nhãn và trả lời được ba câu hỏi ở mục C2.
- ☐ Chia dữ liệu TRƯỚC khi fit vectorizer — kiểm lại thứ tự trong train.py.
- ☐ Không có lệnh fit hay fit_transform nào chạy trên tập val hoặc test.
- ☐ Có script huấn luyện chạy bằng một lệnh, sinh ra reports/metrics.json.
- ☐ Đã chạy phép thử tái lập: hai lần chạy cho hai tệp metrics.json giống nhau.
- ☐ Có cả hai mô hình cơ sở và cả hai con số nằm trong metrics.json.
- ☐ metrics.json có F1 tách theo từng nhãn, không chỉ có chỉ số tổng thể.
- ☐ Hàm làm sạch văn bản giữ dấu tiếng Việt và thay thông tin định danh bằng thẻ.
- ☐ requirements.txt ghi phiên bản chính xác bằng dấu ==.
- ☐ Có ít nhất ba test cho hàm làm sạch văn bản, chạy PASS bằng một lệnh.

So sánh nhanh ba track — bảng đối chiếu để tự kiểm

Tiêu chí	Track SE	Track DA	Track AI
Sản phẩm chạy được là gì	Máy chủ ứng dụng lên được, gọi được các endpoint CRUD	Pipeline chạy một lệnh, sinh bảng staging và bảng reject	Script huấn luyện chạy một lệnh, sinh reports/metrics.json
Ba lớp tách biệt	routes / services / repositories	extract / transform / load	data / features / models
Lớp nào chứa quy tắc quan trọng nhất	services	transform và quality	features và models
Lớp nào phải test kỹ nhất	services (dùng repository giả)	rules.py và checks.py (hàm thuần)	text.py (hàm làm sạch)
Mục kiểm chứng "chạy lại được"	Migration chạy lần hai không lỗi	Chạy lần hai không nhân đôi bản ghi	Hai lần chạy cho metrics.json giống nhau
Lỗi nghiêm trọng nhất của track	Lẫn tầng: service biết về HTTP	Âm thầm bỏ dòng dữ liệu, không có bảng reject	Rò rỉ dữ liệu: fit vectorizer trước khi chia
Câu hỏi thầy hay hỏi nhất ở buổi 8	"Yêu cầu sai định dạng bị chặn ở tầng nào?"	"Chạy lại lần hai thì số bản ghi thế nào?"	"Vì sao em dùng F1 macro mà không dùng độ chính xác?"
Nguyên tắc chung áp dụng cho cả ba	Truy vết được · Tái lập được · Kiểm chứng được	Truy vết được · Tái lập được · Kiểm chứng được	Truy vết được · Tái lập được · Kiểm chứng được

Ba việc làm ngay trong tuần này, trước workshop buổi 8

1. Dựng xong bộ khung thư mục theo mẫu của track mình và commit nó, dù các tệp còn trống. Có khung rồi thì viết code vào chỗ đúng, không phải sửa cấu trúc sau.

2. Viết và chạy được MỘT test cho quy tắc nghiệp vụ quan trọng nhất của em. Một test chạy được có giá trị hơn mười test viết dở.

3. Chạy phép thử "chạy lại được" của track mình và ghi kết quả ra giấy. Nếu chưa đạt, đó là việc số một của em ở buổi 8.

PHẦN D — Dùng chung cho ba track: commit, câu hỏi chuẩn và xử lý sự cố

D1. Thông điệp commit: ví dụ chưa tốt và tốt

Học phần dùng quy ước Conventional Commits: <loại>: <việc đã làm>. Các loại hay dùng: feat (tính năng), fix (sửa lỗi), test (thêm test), docs (tài liệu), refactor (sắp xếp lại mã, không đổi hành vi), chore (việc phụ trợ). Lịch sử commit là minh chứng của CLO6.

Chưa tốt	Tốt	Vì sao
update	feat: thêm endpoint POST /tickets kèm validator	Đọc là biết commit chứa gì, không cần mở diff.
fix bug	fix: QT-09 cho phép tạo phiếu mới khi phiếu cũ đã HUY	Nêu đúng quy tắc và hành vi đã sửa — truy vết được về SRS.
code xong phần 2	test: thêm TC-02 giá trị biên ngày hết hạn bảo hành	Một commit một việc; dễ quay lui nếu test này làm vỡ thứ khác.
final final v2	docs: README bổ sung lệnh chạy migration và seed	Commit tài liệu tách khỏi commit mã nguồn.

D2. Bộ ba câu hỏi chuẩn áp dụng cho code — ví dụ trả lời

Câu hỏi	Câu trả lời yếu	Câu trả lời tốt (ví dụ track SE)
1. Vì sao em chọn cách này mà không phải cách kia?	"Vì trên mạng làm vậy."	"Service ném AppError có mã nghiệp vụ thay vì trả mã HTTP, để cùng một quy tắc dùng được cho API, script và test; routes là nơi duy nhất biết HTTP."
2. Nếu dữ liệu tăng gấp mười thì chỗ nào hỏng trước?	"Chắc không sao."	"findBySerial vẫn nhanh nhờ index idx_tickets_serial. Chỗ yếu là nextTicketCode đếm COUNT(*) theo trung tâm — chậm dần và có thể trùng mã khi hai người tạo phiếu cùng lúc; nên chuyển sang SEQUENCE."
3. Chỉ cho thầy dòng mã chịu trách nhiệm cho việc này.	Giải thích chung chung bằng lời.	Mở ticket.service.js, chỉ vào dòng existing.find(...) và lệnh throw new AppError('PHIEU_DANG_MO', ...) của QT-09.

D3. Lỗi thường gặp khi dựng module cốt lõi và cách xử lý nhanh

Triệu chứng	Nguyên nhân thường gặp	Cách xử lý
ECONNREFUSED 127.0.0.1:5432	PostgreSQL chưa chạy, hoặc sai cổng/host trong .env	Khởi động CSDL; kiểm tra DATABASE_URL; trong docker-compose dùng tên dịch vụ (db) thay cho localhost.
relation "tickets" already exists	Migration thiếu IF NOT EXISTS	Thêm IF NOT EXISTS cho mọi CREATE TABLE/INDEX, hoặc dùng công cụ migration có bảng theo dõi phiên bản.
ModuleNotFoundError: No module named 'src'	Chạy python src/pipeline.py thay vì chạy dạng module	Chạy từ thư mục gốc: python -m src.pipeline; thêm tệp __init__.py vào src/ và các thư mục con.

Triệu chứng	Nguyên nhân thường gặp	Cách xử lý
Test chạy được trên máy em, FAIL trên máy thầy	Test phụ thuộc dữ liệu có sẵn trong CSDL của em, hoặc phụ thuộc ngày giờ hiện tại	Dùng repository giả hoặc fixture tự dựng dữ liệu; truyền tham số now vào hàm có so sánh ngày (như isUnderWarranty).
Hai lần train cho metrics.json khác nhau	Quên random_state ở một chỗ, hoặc thư viện không ghim phiên bản	Truyền hat_giong vào MỌI chỗ ngẫu nhiên; ghim phiên bản bằng ==; dùng sort_keys khi ghi JSON.
Đã push nhưng không thấy tag v0.2 trên GitHub	git push origin main không đẩy tag	Chạy thêm git push origin v0.2 rồi mở trang releases/tag/v0.2 để kiểm tra.

VAN LANG UNIVERSITY · FACULTY OF INFORMATION TECHNOLOGY

SPECIALIZED GRADUATION TOPIC 1 · SEMESTER 1, ACADEMIC YEAR 2026 – 2027

Self-study material, session 7 — Implementing the core module by specialisation

Layering and CRUD on a relational database [SE] · A re-runnable ETL pipeline with quality checks [DA] · Collecting, cleaning and splitting data correctly [AI]. Every part comes with complete sample code that runs on the Mekong Mobile dataset.

This material produces components 1 and 2 of Assignment 2 — a working core module and its test suite. Start at the session 8 workshop; submit at the end of session 12.

Author: **Dr. Nguyễn Trí Hải** · Department of Software Engineering

For internal use · Specialisation self-study set (30% of the session 7 theory time).

Bilingual Vietnamese – English material: this English version follows Part D of the Vietnamese version.

How to use this material

Taught by the lecturer in class (session 7)	Read on your own in this material
What a core module is and how to choose it; layering and one-way dependencies; the three layers of input validation; what "runs with one command" means; commit discipline and commit messages; how to write your first test; the three standard questions applied to code.	How to implement it on your track's stack: folder structure, sample code for each layer, how to write tests, and a self-check list to complete before the session 8 workshop.

Four things that hold for all three tracks — read before your own part

1. LAYERING is mandatory, not a recommendation. Every track must have at least three separate layers: an input layer · a business-logic layer · a data-access layer. Putting everything in one file drops you a rubric level on the "Technical quality" criterion (50% of the assignment mark).

2. RUNS WITH ONE COMMAND. Not "open the notebook and run each cell", not "run this file and then that file". One command, written in the README, that works on another machine.

3. RE-RUNNABLE WITHOUT CORRUPTING DATA. A second run must give the same result as the first: no duplicated records, no "table already exists" errors. This is an Assignment 2 checklist item.

4. TESTS PROVE THE CODE WORKS. At least three tests for the most important business logic, one of them for invalid data. Tests must run with one command and must PASS when the lecturer runs them.

Recap: the scope of the Assignment 2 core module

The core module is the part of the system without which the product no longer makes sense. It is NOT the prettiest part, nor the part with the most code. For the Mekong Mobile case study, the typical core module per track is:

Track	Typical core module	Matching business flow	Why it is the core
SE	Receiving and processing warranty tickets	L2 — receive the device, check the warranty period, assign a technician	Without it the system can no longer serve the main business of the service centre.
DA	Pipeline that cleans and merges customer data	L6 — merge duplicate customers coming from many stores	Every downstream report relies on the cleaned customer data; a mistake here makes everything downstream wrong.
AI	Automatic classification of issue descriptions into fault groups	L10 — suggest a fault group automatically when staff type a description	Without it the product is just a manual entry form with no AI value left.

If the core module you chose in Assignment 1 differs from these three, keep your choice. These examples only benchmark the expected depth and scope.

Illustration: the three layers of all three tracks in one picture

Before reading your own track, look at the figure below. All three tracks share the same shape: the top layer receives input, the middle layer holds the rules, the bottom layer reads and writes data. Arrows only point downward — a lower layer never calls back up.

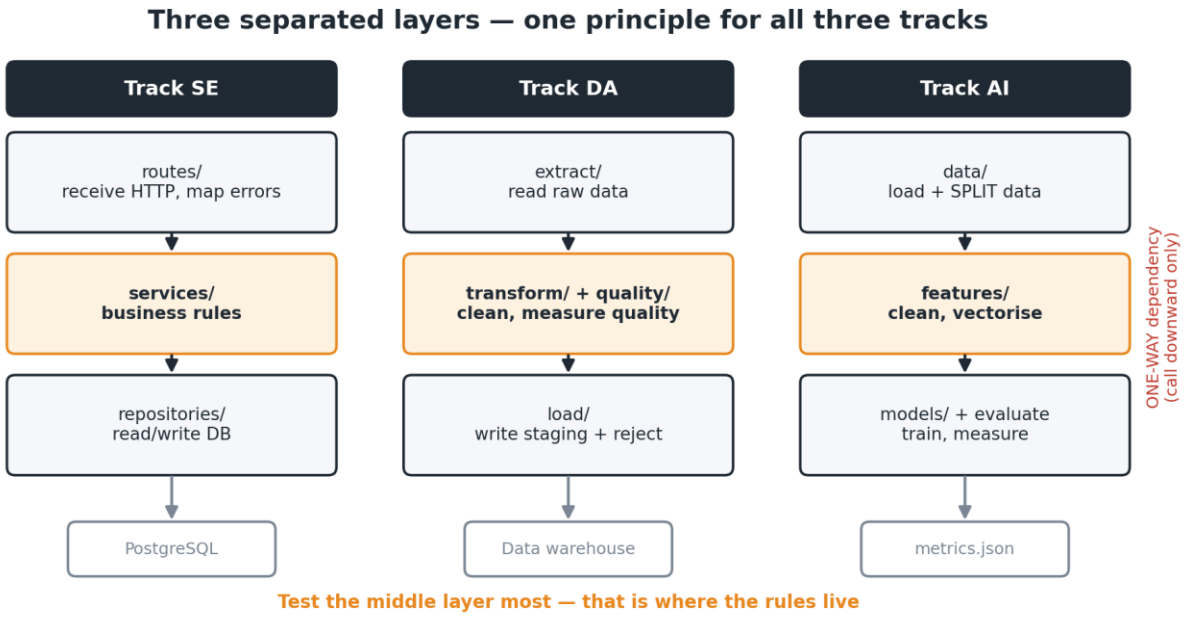


Figure 1. The middle layer (orange) holds the business rules and deserves the most tests. If you cannot point to the folder of your middle layer, your structure is not there yet.

PART A — Track SE: Layering and CRUD on a relational database

A1. Required folder structure

The structure below is the minimum accepted for Assignment 2. Folder names may follow your framework's conventions, but the SEPARATION between the four layers must stay.

Folder structure — example with Node.js + Express (Spring Boot is similar)

```
du-an/
├── src/
│   ├── routes/              # LAYER 1: receives HTTP, no business logic
│   │   └── ticket.routes.js
│   ├── services/            # LAYER 2: business logic, knows NOTHING about HTTP
│   │   └── ticket.service.js
│   ├── repositories/        # LAYER 3: database access, knows NOTHING about business
│   │   └── ticket.repository.js
│   ├── validators/          # input validation
│   │   └── ticket.validator.js
│   ├── errors/              # the application's own error classes
│   │   └── AppError.js
│   └── app.js               # wires the layers together, no logic
├── migrations/              # steps that create and alter tables, run in order
│   ├── 001_create_customers.sql
│   └── 002_create_tickets.sql
├── seeds/                   # sample data so there is something to look at
│   └── 001_seed_demo.sql
├── tests/
│   ├── ticket.service.test.js
│   └── ticket.repository.test.js
├── .env.example             # lists EVERY variable, values are samples
├── package.json
├── package-lock.json        # MUST be committed - this is the version lock file
└── README.md
```

The one-way dependency rule — check it by reading the import statements

routes MAY import services. services MAY import repositories. Never the other way round.

services must NEVER import anything HTTP-related (no req, no res, no HTTP status codes in services).

repositories must NEVER contain business rules. They only read and write.

The 30-second test: open your service file and search for "req", "res", "status(" and "SELECT". If you find any of them, the layers are mixed and must be fixed before session 8.

A2. Migrations — create tables exactly as in the DDL submitted for Assignment 1

A migration is an SQL file (or a migration-tool script) run in order to build the database structure from scratch. Assignment 2 has three requirements for migrations: running on an empty database creates every table · a second run raises no error · it matches the DDL submitted in A1, or any difference is explained in docs/thay-doi-thiet-ke.md.

migrations/002_create_tickets.sql — note IF NOT EXISTS and INDEX

```
-- Warranty ticket table, for business flow L2
CREATE TABLE IF NOT EXISTS tickets (
  ticket_id      BIGSERIAL PRIMARY KEY,
  ticket_code    VARCHAR(20)  NOT NULL UNIQUE,
  customer_id    BIGINT       NOT NULL REFERENCES customers(customer_id),
  serial_no      VARCHAR(40)  NOT NULL,
  product_id     BIGINT       NOT NULL REFERENCES products(product_id),
  center_id      INT          NOT NULL REFERENCES service_centers(center_id),
  category_id    INT          NOT NULL REFERENCES issue_categories(category_id),
  priority       VARCHAR(12)  NOT NULL,
```

```
  status         VARCHAR(20)  NOT NULL DEFAULT 'MOI_TIEP_NHAN',
  technician_id  BIGINT       NULL REFERENCES technicians(technician_id),
  is_warranty    BOOLEAN      NOT NULL,
  received_at    TIMESTAMP    NOT NULL DEFAULT NOW(),
  due_date       TIMESTAMP    NOT NULL,
  closed_at      TIMESTAMP    NULL,
  CONSTRAINT chk_priority CHECK (priority IN ('CAO','TRUNG_BINH','THAP')),
  CONSTRAINT chk_closed   CHECK (closed_at IS NULL OR closed_at >= received_at)
);

-- INDEXES on the columns used for FILTERING and SORTING.
-- This is where the performance non-functional requirement becomes real design.
CREATE INDEX IF NOT EXISTS idx_tickets_serial   ON tickets(serial_no);
CREATE INDEX IF NOT EXISTS idx_tickets_status   ON tickets(status);
CREATE INDEX IF NOT EXISTS idx_tickets_center_r ON tickets(center_id, received_at DESC);
```

Three migration mistakes that stop the lecturer from rebuilding your system

Missing IF NOT EXISTS. The second run fails with "relation already exists" and the whole script stops. This violates the REPRODUCIBLE principle.

Migrations with no clear order. Prefix names with numbers (001_, 002_) and state the run order in the README, or use your framework's migration tool.

Tables only, no sample data. The system comes up empty and the grader has nothing to assess. Provide a seed file that loads at least 20–30 records per main table.

A3. The three layers of one business operation — complete sample code for flow L2

Below are all three layers for one real operation: receiving a device for warranty service. Read the responsibility boundary of each layer carefully — that is what this material teaches, not library syntax.

Layer 3 — Repository: reads and writes, decides nothing

src/repositories/ticket.repository.js

```
// This layer knows ONLY the database. No business rules live here.
class TicketRepository {
  constructor(db) { this.db = db; }

  async findBySerial(serialNo) {
    const { rows } = await this.db.query(
      `SELECT t.*, p.warranty_months, p.product_name
         FROM tickets t JOIN products p ON p.product_id = t.product_id
        WHERE t.serial_no = $1
        ORDER BY t.received_at DESC`, [serialNo]);
    return rows;
  }

  async findOpenByTechnician(technicianId) {
    const { rows } = await this.db.query(
      `SELECT ticket_id FROM tickets
        WHERE technician_id = $1 AND status NOT IN ('HOAN_TAT','HUY')`,
      [technicianId]);
    return rows;
  }

  async insert(data) {
    const { rows } = await this.db.query(
      `INSERT INTO tickets (ticket_code, customer_id, serial_no, product_id,
          center_id, category_id, priority, is_warranty, due_date)
       VALUES ($1,$2,$3,$4,$5,$6,$7,$8,$9) RETURNING *`,
      [data.ticketCode, data.customerId, data.serialNo, data.productId,
       data.centerId, data.categoryId, data.priority, data.isWarranty, data.dueDate]);
    return rows[0];
  }
```

```
  async updateTechnician(ticketId, technicianId) {
    const { rows } = await this.db.query(
      'UPDATE tickets SET technician_id = $2 WHERE ticket_id = $1 RETURNING *',
      [ticketId, technicianId]);
    return rows[0];
  }
  // Generates ticket code PB-<centre id>-<5-digit sequence>, e.g. PB-2-00007.
  // Fine for a prototype; a system with concurrent users should use a DB SEQUENCE.
  async nextTicketCode(centerId) {
    const { rows } = await this.db.query(
      'SELECT COUNT(*)::int AS n FROM tickets WHERE center_id = $1', [centerId]);
    return `PB-${centerId}-${String(rows[0].n + 1).padStart(5, '0')}`;
  }
}
module.exports = TicketRepository;
```

Layer 2 — Service: all the business rules, knows nothing about HTTP

src/services/ticket.service.js — note: no req, no res, no 400/409 numbers

```
const { AppError } = require('../errors/AppError');

// SLA by priority, in hours - business rule QT-08
const SLA_HOURS = { CAO: 24, TRUNG_BINH: 72, THAP: 120 };
const MAX_OPEN_PER_TECH = 8;             // business rule QT-11

class TicketService {
  constructor(ticketRepo, customerRepo, techRepo) {
    this.tickets = ticketRepo;
    this.customers = customerRepo;
    this.techs = techRepo;
  }

  // QT-05: a device is under warranty until warranty_months after the sale date
  isUnderWarranty(soldAt, warrantyMonths, now = new Date()) {
    const expiry = new Date(soldAt);
    expiry.setMonth(expiry.getMonth() + warrantyMonths);
    return now <= expiry;
  }

  async receiveDevice(input) {
    // 1. Existence check - a business rule, not a format check
    const customer = await this.customers.findById(input.customerId);
    if (!customer) throw new AppError('KHACH_KHONG_TON_TAI',
      'No customer found with the given id.');

    // 2. QT-09: one serial may not have two open tickets at the same time
    const existing = await this.tickets.findBySerial(input.serialNo);
    const openOne = existing.find(t => !['HOAN_TAT','HUY'].includes(t.status));
    if (openOne) throw new AppError('PHIEU_DANG_MO',
      `Device ${input.serialNo} already has ticket ${openOne.ticket_code} in progress.`);

    // 3. Work out the warranty status
    const isWarranty = this.isUnderWarranty(
      input.soldAt, input.warrantyMonths);

    // 4. Work out the due date from the SLA
    const hours = SLA_HOURS[input.priority];
    if (!hours) throw new AppError('UU_TIEN_KHONG_HOP_LE',
      'Priority must be CAO, TRUNG_BINH or THAP.');
    const dueDate = new Date(Date.now() + hours * 3600 * 1000);

    // 5. Create the ticket
    return this.tickets.insert({ ...input, isWarranty, dueDate,
      ticketCode: await this.tickets.nextTicketCode(input.centerId) });
  }

  // QT-11: never assign more than MAX_OPEN_PER_TECH open tickets to one technician
  async assignTechnician(ticketId, technicianId) {
    const tech = await this.techs.findById(technicianId);
    if (!tech || !tech.is_active) throw new AppError('KTV_KHONG_KHA_DUNG',
      'The technician does not exist or is no longer active.');
    const open = await this.tickets.findOpenByTechnician(technicianId);
```

```
    if (open.length >= MAX_OPEN_PER_TECH) throw new AppError('KTV_QUA_TAI',
      `The technician has ${open.length} open tickets, over the limit of ${MAX_OPEN_PER_TECH}.`);
    return this.tickets.updateTechnician(ticketId, technicianId);
  }
}
module.exports = TicketService;
```

Why the service must not know HTTP status codes

The same business rule can be called from a REST API, a command-line command, a scheduled job or a test. Only the REST API has HTTP status codes.

The service throws an error with a BUSINESS CODE (PHIEU_DANG_MO, KTV_QUA_TAI). The routes layer translates that business code into the matching HTTP status.

This lets you test every business rule without starting a web server — which is what makes tests fast and stable.

This is also your answer to standard question no. 1 ("why did you choose this approach?") when the lecturer asks about your architecture in session 8.

Layer 1 — Routes: translates between HTTP and business, no logic

src/routes/ticket.routes.js

```
const express = require('express');
const { validateReceive } = require('../validators/ticket.validator');

// Map BUSINESS CODE -> HTTP STATUS. Everything that "knows HTTP" lives here.
const HTTP_BY_CODE = {
  DU_LIEU_KHONG_HOP_LE:  400,   // layer 1: bad format, with the list of fields
  KHACH_KHONG_TON_TAI:   404,
  PHIEU_DANG_MO:         409,   // state conflict
  UU_TIEN_KHONG_HOP_LE:  400,
  KTV_KHONG_KHA_DUNG:    404,
  KTV_QUA_TAI:           422,   // syntactically valid, cannot be processed by the business
};

module.exports = function ticketRoutes(ticketService) {
  const r = express.Router();

  r.post('/tickets', async (req, res, next) => {
    try {
      const input = validateReceive(req.body);        // format-check layer
      const ticket = await ticketService.receiveDevice(input);
      res.status(201).json({ data: ticket });
    } catch (e) {
      if (e.code && HTTP_BY_CODE[e.code])
        return res.status(HTTP_BY_CODE[e.code]).json({ error: e.code, message: e.message, chi_tiet:
e.chiTiet });
      next(e);                                        // unexpected error -> 500
    }
  });

  return r;
};
```

Two supporting files that make the sample run: AppError and the validator

The three layers above rely on two files not yet shown: the AppError class and the validateReceive function. The short versions below are enough to make the layers run; the reference repo mekong-se-baohanh in the course materials folder has a fuller version.

src/errors/AppError.js

```
// src/errors/AppError.js — a business error with a CODE, no HTTP status inside
```

```
class AppError extends Error {
  constructor(code, message, chiTiet = undefined) {
    super(message);
    this.name = 'AppError';
    this.code = code;              // e.g. 'PHIEU_DANG_MO' — routes map it to HTTP
    if (chiTiet !== undefined) this.chiTiet = chiTiet;   // e.g. list of invalid fields
  }
}
module.exports = { AppError };
```

src/validators/ticket.validator.js

```
// src/validators/ticket.validator.js — LAYER 1: format and type, NO database lookup
const { AppError } = require('../errors/AppError');
const UU_TIEN = ['CAO', 'TRUNG_BINH', 'THAP'];
const RE_SERIAL = /^[A-Za-z0-9-]{6,40}$/;

function validateReceive(body = {}) {
  const loi = [];                                  // collect ALL errors, report once
  for (const f of ['customerId', 'productId', 'centerId', 'categoryId'])
    if (!Number.isInteger(body[f]) || body[f] <= 0)
      loi.push({ truong: f, ly_do: 'must be a positive integer' });
  const serial = typeof body.serialNo === 'string' ? body.serialNo.trim() : '';
  if (!RE_SERIAL.test(serial))
    loi.push({ truong: 'serialNo', ly_do: 'letters, digits, hyphen; 6–40 characters' });
  if (!UU_TIEN.includes(body.priority))
    loi.push({ truong: 'priority', ly_do: 'must be CAO, TRUNG_BINH or THAP' });
  if (!/^\d{4}-\d{2}-\d{2}$/.test(body.soldAt || '') || Number.isNaN(Date.parse(body.soldAt)))
    loi.push({ truong: 'soldAt', ly_do: 'must use the YYYY-MM-DD format' });
  if (!Number.isInteger(body.warrantyMonths) || body.warrantyMonths < 0)
    loi.push({ truong: 'warrantyMonths', ly_do: 'must be a non-negative integer' });
  if (loi.length) throw new AppError('DU_LIEU_KHONG_HOP_LE',
    `Invalid input in fields: ${loi.map(l => l.truong).join(', ')}`, loi);
  return { ...body, serialNo: serial };            // return the NORMALISED data
}
module.exports = { validateReceive };
```

Three things to notice in the validator

Collect every error, report once. The user fixes everything in one go instead of resubmitting field by field.

No database lookups. "Does the customer exist?" is a business question and belongs to layer 2 (service).

Return normalised data. The service receives a trimmed serial and never has to check it again.

Illustration: the path of one request and where each kind of error is stopped

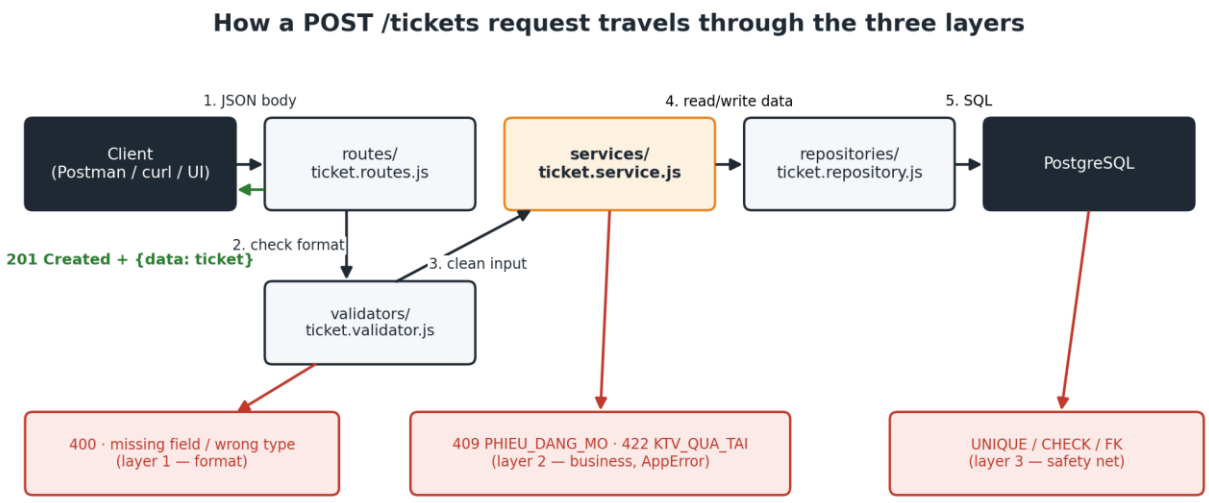


Figure 2. Format errors stop at the validator (400), business errors stop at the service (409/422), database constraints are the last safety net. Only routes know HTTP codes.

Example: calling the API with curl and reading the response

Three calls — three layers reacting differently

```
# (1) Happy path -> 201 Created
curl -i -X POST http://localhost:3000/tickets -H 'Content-Type: application/json' \
  -d '{"customerId":1,"serialNo":"SN-10293","productId":5,"centerId":2,
       "categoryId":1,"priority":"CAO","soldAt":"2026-03-01","warrantyMonths":12}'
HTTP/1.1 201 Created
{"data":{"ticket_id":101,"ticket_code":"PB-2-00007","status":"MOI_TIEP_NHAN",
          "is_warranty":true,"due_date":"2026-10-13T09:15:00.000Z", ...}}

# (2) priority missing, serial too short -> 400, blocked by layer 1, DB NOT touched
HTTP/1.1 400 Bad Request
{"error":"DU_LIEU_KHONG_HOP_LE",
 "message":"Invalid input in fields: serialNo, priority",
 "chi_tiet":[{"truong":"serialNo","ly_do":"letters, digits, hyphen; 6–40 characters"},
             {"truong":"priority","ly_do":"must be CAO, TRUNG_BINH or THAP"}]}

# (3) Send request (1) again -> 409, blocked by layer 2 (rule QT-09)
HTTP/1.1 409 Conflict
{"error":"PHIEU_DANG_MO","message":"Device SN-10293 already has ticket PB-2-00007 in progress."}
```

Match these with the screenshots you must submit

Call (1) proves the happy path; (2) proves layer 1 names the invalid FIELDS; (3) proves a real business rule (QT-09). Put these three screenshots straight into section 3 of your Assignment 2 PDF.

An error response never contains a stack trace, table names or SQL. If you see any of those, routes is leaking raw errors.

A4. Three layers of input validation — each catches a different kind of error

Layer	What it checks	Where in the code	Concrete example
1. Format and type	Required fields present · correct type · valid length · value within the allowed set	validators/, runs before the service	serial_no is a 6–40 character string (letters, digits, hyphen) · priority is in {CAO, TRUNG_BINH, THAP} · customer_id is a positive integer
2. Business rules	Conditions that can only be known by looking up existing data	services/, throws an AppError with a business code	This serial has no open ticket · the technician is not overloaded · the customer exists
3. Database constraints	The last safety net, catching even cases where the code is buggy	migrations/, via UNIQUE, CHECK, FOREIGN KEY, NOT NULL	ticket_code UNIQUE · priority CHECK · closed_at >= received_at

Why all three layers are needed instead of just one

Layer 1 only: the system accepts well-formed but business-invalid data — two open tickets can be created for the same device.

Layer 2 only: every malformed request has to hit the database before being rejected, wasting resources and producing vague error messages.

Layer 3 only: users receive raw database errors — hard to understand and revealing the system's internals, which is a security problem.

Layer	What it checks	Where in the code	Concrete example

The question you will be asked in session 8: "If I send a create-ticket request with an empty priority, which of your layers stops it, and what message does the user get?" Prepare to answer by pointing at the code.

A5. Unit tests for the service layer — using fake repositories

Testing the service layer is where effort pays off most, because that is where the business rules live. Since the service knows nothing about HTTP and receives its repositories through the constructor, you replace the real repository with a fake object and the tests run in milliseconds, with no database.

tests/ticket.service.test.js — the three tests required for Assignment 2

```
const TicketService = require('../src/services/ticket.service');

// Fake repositories: return exactly the data we want, no database connection
const fakeCustomers = { findById: async id => (id === 1 ? { customer_id: 1 } : null) };
const fakeTechs     = { findById: async id => ({ technician_id: id, is_active: true }) };

function makeTickets(openTickets = [], existingForSerial = []) {
  return {
    findBySerial:          async () => existingForSerial,
    findOpenByTechnician:  async () => openTickets,
    insert:                async d  => ({ ticket_id: 99, ...d }),
    updateTechnician:      async () => ({ ok: true }),
    nextTicketCode:        async () => 'PB-2-00007',
  };
}

test('TC-01 happy path: a device still under warranty gets a ticket', async () => {
  const svc = new TicketService(makeTickets(), fakeCustomers, fakeTechs);
  const t = await svc.receiveDevice({
    customerId: 1, serialNo: 'SN-10293', productId: 5, centerId: 2,
    categoryId: 1, priority: 'CAO',
    soldAt: '2026-03-01', warrantyMonths: 12,
  });
  expect(t.isWarranty).toBe(true);
  expect(t.ticketCode).toBe('PB-2-00007');
  expect(t.dueDate.getTime()).toBeGreaterThan(Date.now());
});

test('TC-02 boundary: last day still covered, next day expired', () => {
  const svc = new TicketService(makeTickets(), fakeCustomers, fakeTechs);
  // sold 01/03/2026, 12-month warranty -> expires 01/03/2027
  expect(svc.isUnderWarranty('2026-03-01', 12, new Date('2027-03-01'))).toBe(true);
  expect(svc.isUnderWarranty('2026-03-01', 12, new Date('2027-03-02'))).toBe(false);
});

test('TC-03 invalid data: a serial with an open ticket is rejected', async () => {
  const svc = new TicketService(
    makeTickets([], [{ ticket_code: 'PB-2-00007', status: 'DANG_SUA' }]),
    fakeCustomers, fakeTechs);
  await expect(svc.receiveDevice({
    customerId: 1, serialNo: 'SN-10293', productId: 5, centerId: 2,
    categoryId: 1, priority: 'CAO', soldAt: '2026-03-01', warrantyMonths: 12,
  })).rejects.toMatchObject({ code: 'PHIEU_DANG_MO' });
});
```

These three tests match the three test types Assignment 2 requires

TC-01 — happy path: proves the feature works under normal conditions.

TC-02 — boundary value: the last day under warranty and the first day after it. In practice this type of test catches the most bugs.

TC-03 — invalid data: proves the system REJECTS correctly, with a specific error code instead of crashing.

How to run: `npm test` (in package.json: "test": "jest") — write exactly this command in the README. The lecturer will run that very command.

Example: expected output of npm test

Terminal

```
$ npm test
> mekong-baohanh@0.2.0 test
> jest

 PASS  tests/ticket.service.test.js
  ✓ TC-01 happy path: a device still under warranty gets a ticket (4 ms)
  ✓ TC-02 boundary: last day still covered, next day expired (1 ms)
  ✓ TC-03 invalid data: a serial with an open ticket is rejected (1 ms)

Test Suites: 1 passed, 1 total
Tests:       3 passed, 3 total
Time:        0.41 s
```

This screen is the evidence for checklist item C4

Capture the whole screen; the line "Tests: 3 passed" must be visible. Name tests after the rule, as above — the grader learns what you checked just by reading the names.

A6. Self-check list before the session 8 workshop [SE]

- ☐ Works from an empty database: run migrations, then seeds, then start the app, with no errors.
- ☐ Running migrations a second time raises no error (IF NOT EXISTS, or a migration tool handles it).
- ☐ At least two entities with all four operations: create, read, update, delete.
- ☐ Opening the service file you find no "req", "res", "status(" or "SELECT".
- ☐ Opening the repository file you find no business rule (no if checking a business condition).
- ☐ There is a separate validator file, and a request missing a required field returns 400 with a message naming the missing field.
- ☐ At least three unit tests for the service, passing with one command.
- ☐ package-lock.json (or an equivalent version lock file) is committed.
- ☐ The real .env is NOT in the repo; .env.example lists every variable.
- ☐ The README gives the exact commands: install, run migrations, run seeds, start, run tests.

PART B — Track DA: A re-runnable ETL pipeline with quality checks

B1. Required folder structure

Folder structure for an ETL pipeline

```
du-an/
├── src/
│   ├── extract/            # reads raw data, transforms NOTHING
│   │   └── read_sources.py
│   ├── transform/          # cleans and transforms, NO database reads/writes
│   │   ├── clean_customers.py
│   │   └── rules.py        # shared normalisation rules
│   ├── quality/            # quality checks, one function per rule
│   │   └── checks.py
│   ├── load/               # writes to the target, transforms NOTHING
│   │   └── load_staging.py
│   └── pipeline.py         # orchestrates: extract -> transform -> quality -> load
├── sql/
│   ├── 001_create_staging.sql
│   └── 002_create_reject.sql
├── config/
│   └── config.example.yaml  # source paths, quality thresholds
├── tests/
│   ├── test_rules.py
│   └── test_checks.py
├── data/                   # only a .gitkeep in the repo, NEVER commit data
├── reports/                # quality report generated after every run
├── requirements.txt        # EXACT versions, never >=
└── README.md
```

The three-phase boundary — violating it costs marks on Technical quality

extract must NOT transform data. It only reads and returns raw data. If extract also fixes date formats while reading, you no longer know what the original data looked like.

transform must NOT read or write the database. It takes data in and returns data out. That lets you test it without a database — just like the service layer in track SE.

load must NOT transform data. It only writes. If load also modifies data, your quality report no longer reflects what was actually written.

The test: in the transform files search for "connect", "cursor", "to_sql", "read_sql". In the load files search for "replace", "strip", "astype". If you find any, the boundary has been violated.

B2. Normalising the real Mekong Mobile data

The dataset you received has deliberately planted quality problems. The table below lists exactly what you will meet and in which file — you do not need to waste time discovering them.

Problem	File and column	What it looks like	Suggested handling
Four phone-number formats	customers_raw.csv · so_dien_thoai	0763247885 · 0760.833.592 · 076 083 3592 · +84760833592	Strip every non-digit character, turn the +84 prefix into 0, check that exactly 10 digits remain.
Three date formats	orders_2024_2026.csv · ngay	17/08/2025 (62%) · 17-8-25 (24%, two-digit year, no zero padding) · 2025.08.17 (14%) · 0.44% empty cells	Try the three formats %d/%m/%Y · %d-%m-%y · %Y.%m.%d in order of frequency; rows matching none go to the reject table. Watch the second format: two-digit year and no zero

Problem	File and column	What it looks like	Suggested handling
			padding — it is the easiest one to misread.
Missing phone number	customers_raw.csv · so_dien_thoai	about 6.1% of rows are empty	Do NOT drop the row. Flag and count it — an empty phone number is still a real customer.
Duplicate customers	customers_raw.csv	about 20% of records duplicate another record, differing in name and phone formatting	Merge on a composite key: normalised phone + normalised name (diacritics removed, lower case, collapsed spaces).
Amounts that do not match	order_items.csv	about 1.5% of rows have thanh_tien ≠ so_luong × don_gia	Send them to the reject table with both values. NEVER fix the amount yourself — monetary data must be decided by someone with authority.
Order ids repeated across stores	orders_2024_2026.csv · ma_don	about 1,100 order ids appear in several different stores	The real key is the pair (ma_cua_hang, ma_don), not ma_don alone. Record this finding in the Data dictionary.

B3. Sample code — normalisation and quality checks

src/transform/rules.py — every rule is a pure function, testable straight away

```
import re
import unicodedata
from datetime import datetime

DATE_FORMATS = ['%d/%m/%Y', '%d-%m-%y', '%Y.%m.%d']   # in order of frequency

def chuan_hoa_dien_thoai(raw):
    """Return a 10-digit number starting with 0, or None if invalid."""
    if raw is None or str(raw).strip() == '':
        return None
    digits = re.sub(r'\D', '', str(raw))
    if digits.startswith('84') and len(digits) == 11:
        digits = '0' + digits[2:]
    return digits if len(digits) == 10 and digits.startswith('0') else None

def chuan_hoa_ten(raw):
    """Remove diacritics, lower-case, collapse spaces - used AS A MERGE KEY."""
    if raw is None:
        return ''
    s = unicodedata.normalize('NFD', str(raw))
    s = ''.join(c for c in s if unicodedata.category(c) != 'Mn')
    return re.sub(r'\s+', ' ', s).strip().lower()

def doc_ngay(raw):
    """Try the three formats in turn. Return None if none matches."""
    s = str(raw).strip()
    for fmt in DATE_FORMATS:
        try:
            return datetime.strptime(s, fmt).date()
        except ValueError:
            continue
    return None

def khoa_hop_nhat(ho_ten, so_dien_thoai):
    """Composite key that groups the records of the same customer."""
    return (chuan_hoa_dien_thoai(so_dien_thoai) or 'KHONG_SDT',
            chuan_hoa_ten(ho_ten))
```

src/quality/checks.py — every rule returns ONE MEASURABLE NUMBER

```
from dataclasses import dataclass

@dataclass
class KetQuaKiem:
    ma: str              # CL1..CL5
    ten: str
    so_dong_kiem: int
    so_dong_loi: int
    nguong_cho_phep: float   # maximum allowed error rate
    @property
    def ti_le_loi(self):
        return self.so_dong_loi / self.so_dong_kiem if self.so_dong_kiem else 0.0
    @property
    def dat(self):
        return self.ti_le_loi <= self.nguong_cho_phep

def cl1_dien_thoai_hop_le(df, nguong=0.10):
    loi = df['sdt_chuan'].isna() & df['so_dien_thoai'].notna() \
          & (df['so_dien_thoai'].astype(str).str.strip() != '')
    return KetQuaKiem('CL1', 'Phone number can be normalised',
                      len(df), int(loi.sum()), nguong)

def cl2_ngay_hop_le(df, nguong=0.01):
    loi = df['ngay_chuan'].isna()
    return KetQuaKiem('CL2', 'Date parses with one of three formats',
                      len(df), int(loi.sum()), nguong)

def cl3_tien_khop(df, nguong=0.02):
    loi = (df['thanh_tien'] - df['so_luong'] * df['don_gia']).abs() > 1
    return KetQuaKiem('CL3', 'thanh_tien = so_luong x don_gia',
                      len(df), int(loi.sum()), nguong)

def cl4_khoa_khong_trung(df, nguong=0.0):
    loi = df.duplicated(subset=['ma_cua_hang', 'ma_don'], keep=False)
    return KetQuaKiem('CL4', 'Pair (ma_cua_hang, ma_don) is unique',
                      len(df), int(loi.sum()), nguong)

def cl5_khoa_ngoai_ton_tai(df_items, df_orders, nguong=0.0):
    hop_le = df_items['order_id'].isin(df_orders['order_id'])
    return KetQuaKiem('CL5', 'Every order line points to a real order',
                      len(df_items), int((~hop_le).sum()), nguong)
```

Why every quality check must return a NUMBER, not True/False

A number tells you the severity: 3 bad rows out of 26,000 is completely different from 8,000 bad rows out of 26,000, even though both "fail".

With a number you can set an acceptance threshold, and that threshold is the measurable non-functional requirement from Assignment 1 turned into code.

With a number you can compare runs and notice when source data quality gets worse.

This is the VERIFIABLE principle applied to track DA. The quality report generated after every run is your evidence when Assignment 2 is graded.

Example: the three test types for rules.py

tests/test_rules.py

```
# tests/test_rules.py — run with: pytest -q
from datetime import date
import pytest
from src.transform import rules

# TYPE 1 — happy path: all four phone formats that REALLY occur in the data
@pytest.mark.parametrize('tho, mong_doi', [
    ('0763247885',    '0763247885'),
    ('0760.833.592', '0760833592'),
    ('076 083 3592', '0760833592'),
    ('+84760833592', '0760833592'),
```

```
])
def test_TR01_chuan_hoa_dien_thoai(tho, mong_doi):
    assert rules.chuan_hoa_dien_thoai(tho) == mong_doi


# TYPE 1 – three date formats, including the 2-digit-year, unpadded one
@pytest.mark.parametrize('tho', ['17/08/2025', '17-8-25', '2025.08.17'])
def test_TR02_doc_ba_khuon_ngay(tho):
    assert rules.doc_ngay(tho) == date(2025, 8, 17)


# TYPE 2 – boundary: 9 and 11 digits are INVALID, only 10 digits pass
@pytest.mark.parametrize('tho', ['076083359', '07608335921'])
def test_TR03_bien_do_dai(tho):
    assert rules.chuan_hoa_dien_thoai(tho) is None


# TYPE 3 – invalid data: empty or junk returns None, NO exception raised
@pytest.mark.parametrize('tho', [None, '', '   ', 'abc'])
def test_TR04_du_lieu_rac(tho):
    assert rules.chuan_hoa_dien_thoai(tho) is None
    assert rules.doc_ngay(tho) is None
```

Why parametrize

One test function, many data sets: pytest counts each set as a separate test. The inputs are exactly the formats listed in table B2 — the tests reflect real data, not data you invented.

Example: the quality report produced after one run

Code	Rule	Rows checked	Failed rows · rate	Threshold · verdict
CL1	Phone number normalisable	53,812	0 · 0.00%	10% · PASS
CL2	Date parses with one of three formats	26,140	115 · 0.44%	1% · PASS
CL3	thanh_tien = so_luong × don_gia	71,905	1,079 · 1.50%	2% · PASS
CL4	Pair (ma_cua_hang, ma_don) is unique	26,140	0 · 0.00%	0% · PASS
CL5	Order line points to a real order	71,905	37 · 0.05%	0% · FAIL

Illustrative table; your real numbers depend on your data and cleaning rules. Note row CL5: a FAILED rule must still be reported honestly, together with the number of rows moved to the reject table.

B4. Making the pipeline re-runnable — two techniques, pick one

Technique	How it works	When to use	Drawback
Delete and reload by batch	Each run gets a batch id (batch_id). Before loading, delete every row of the same day's batch, then load afresh.	Daily or periodic data; moderate volume.	Slower with large data; the delete condition needs care.
Conditional write by key	Use an insert that handles key conflicts (INSERT ... ON CONFLICT DO UPDATE / MERGE) on the business key.	There is a clear, stable business key.	The business key must be right; a wrong key duplicates data or wrongly overwrites it.

The re-run test — do it before the session 8 workshop

```
# Run 1
python -m src.pipeline --config config/config.yaml
psql -c "SELECT COUNT(*) FROM staging_customers;"   # note this number, e.g. 53812
```

Technique	How it works	When to use	Drawback

```
# Run 2 with EXACTLY the same input, deleting NOTHING by hand
python -m src.pipeline --config config/config.yaml
psql -c "SELECT COUNT(*) FROM staging_customers;"   # MUST be exactly 53812

# If run 2 = 2 x run 1          -> the pipeline is not re-runnable, fix it
# If run 2 raises a key conflict -> the pipeline is not re-runnable, fix it
```

Example: pipeline.py — one command, one batch, no duplicates on re-run

src/pipeline.py

```
# src/pipeline.py — run with: python -m src.pipeline --config config/config.yaml
import argparse, datetime, yaml
from src.extract.read_sources import doc_khach_hang
from src.transform.clean_customers import lam_sach_khach_hang
from src.quality import checks
from src.load.load_staging import ghi_lo, ghi_reject, ghi_bao_cao

def main():
    ap = argparse.ArgumentParser(); ap.add_argument('--config', required=True)
    cfg = yaml.safe_load(open(ap.parse_args().config, encoding='utf-8'))
    batch_id = 'khach-hang-' + datetime.date.today().isoformat()  # one batch per day

    tho = doc_khach_hang(cfg['nguon']['khach_hang'])          # 1. EXTRACT: read only
    sach, reject = lam_sach_khach_hang(tho)                     # 2. TRANSFORM: pure function
    ket_qua = [checks.cl1_dien_thoai_hop_le(sach)]              # 3. QUALITY: returns NUMBERS

    ghi_lo(sach, batch_id)        # 4. LOAD: DELETE ... WHERE batch_id = %s then INSERT
    ghi_reject(reject, batch_id)  #    in one transaction -> re-runs never duplicate
    ghi_bao_cao(ket_qua, batch_id, so_dong_vao=len(tho), so_reject=len(reject))
    print(f'{batch_id}: in {len(tho)} · clean {len(sach)} · reject {len(reject)}')

if __name__ == '__main__':
    main()
```

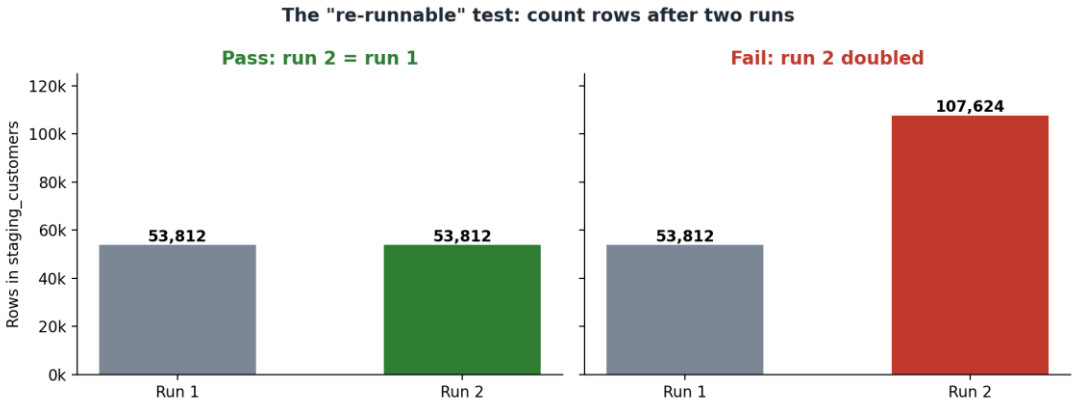


Figure 3. Left: a passing pipeline — the second run yields exactly the same row count. Right: a pipeline that only INSERTs, without deleting the old batch or handling key conflicts, doubles the rows.

B5. The reject table — where unprocessable rows go

The reject table is a mandatory Assignment 2 component for track DA. The principle: NEVER silently drop a row. Every row that cannot be processed must end up somewhere, with a reason, so others can trace it.

sql/002_create_reject.sql

```
CREATE TABLE IF NOT EXISTS reject_rows (
  reject_id    BIGSERIAL PRIMARY KEY,
  batch_id     VARCHAR(40) NOT NULL,      -- which run batch
  nguon        VARCHAR(60) NOT NULL,      -- source file name
  so_dong      INT         NOT NULL,      -- row number in the source file
```

```
  ma_quy_tac   VARCHAR(10) NOT NULL,      -- CL1..CL5 - which rule rejected it
  ly_do        TEXT        NOT NULL,      -- written for HUMANS, not for machines
  du_lieu_goc  JSONB       NOT NULL,      -- the whole original row, for tracing
  tao_luc      TIMESTAMP   NOT NULL DEFAULT NOW()
);
CREATE INDEX IF NOT EXISTS idx_reject_batch ON reject_rows(batch_id, ma_quy_tac);
```

Three requirements for the reject table when Assignment 2 is graded

The reason column is written for people: "thanh_tien 11990000 differs from so_luong 2 x don_gia 5990000 = 11980000", not "CL3 failed".

The original-data column keeps the entire source row, so others can verify the pipeline's conclusion.

The quality report at the end of each run states the number of rejected rows per rule, and the total must match the rows in the reject table for that batch.

B6. Self-check list before the session 8 workshop [DA]

- ☐ The pipeline runs with ONE command, written in the README. Not a notebook run cell by cell.
- ☐ The re-run test has been done: the second run gives the same record count as the first.
- ☐ The three phases extract / transform / load live in three different files and never call in the wrong direction.
- ☐ At least five quality-check functions CL1–CL5, each returning the number and rate of failed rows.
- ☐ There is a reject table and it really contains rows after a run (the dataset certainly has bad rows).
- ☐ A quality report is generated automatically into reports/ after every run.
- ☐ requirements.txt pins exact versions (== rather than >=).
- ☐ No data files are committed; only .gitkeep and the source paths in the config.
- ☐ At least three tests for the normalisation functions in rules.py, passing with one command.
- ☐ The finding "the real key is the pair (ma_cua_hang, ma_don)" is recorded in the Data dictionary.

PART C — Track AI: Collecting, cleaning and splitting data correctly

C1. Required folder structure

Folder structure for a machine-learning project

```
du-an/
├── notebooks/
│   └── 01_eda.ipynb          # data EXPLORATION - NOT a deliverable
├── src/
│   ├── data/
│   │   ├── load.py           # loads raw data
│   │   └── split.py          # SPLITS data - done BEFORE any transformation
│   ├── features/
│   │   └── text.py           # text cleaning, vectorisation
│   ├── models/
│   │   ├── baseline.py       # MANDATORY baseline models for comparison
│   │   └── train.py          # trains the main model
│   └── evaluate.py           # measures and records results
├── artifacts/                # saved model, vectorizer (do not commit large files)
├── reports/
│   └── metrics.json          # measured results, generated automatically
├── configs/
│   └── train.yaml            # hyperparameters, random seed
├── tests/
│   └── test_text.py
├── requirements.txt
└── README.md
```

A notebook is not a deliverable

Notebooks are for EXPLORING: looking at the label distribution, a few samples, a quick idea. That is legitimate and necessary.

But "retraining the model" must be a script that runs with one command and gives the same result every time. A notebook run cell by cell cannot guarantee that because the execution order is undefined.

Assignment 2 checklist item for track AI: the command `python -m src.models.train --config configs/train.yaml` must run and produce reports/metrics.json.

You may submit the EDA notebook as well — it is evidence that you understood the data before modelling.

C2. The first thing to do: look at the label distribution

Before anything else, count the samples of each label. It takes 30 seconds and determines everything that follows: which metric to use, whether imbalance needs handling, what the baseline should be.

The first step in the EDA notebook

```
import pandas as pd

df = pd.read_csv('data/issue_descriptions.csv')
print(df.shape)                                    # (4000, 3)
print(df['issue_category'].value_counts())
print(df['issue_category'].value_counts(normalize=True).round(3))

# Questions you must answer BEFORE going further:
#  1. Which label is the most frequent, and what share does it have?
#     -> that is the accuracy of an "always predict the majority" model.
#        If your model does not beat this number, it is useless.
#  2. Which label is the rarest, and are there enough samples to learn it?
```

```
#     -> below 50 samples per label, measurements are very unstable.
#  3. Is there a catch-all label (such as KHAC = "other")?
#     -> catch-all labels usually drag scores down and must be discussed in the report.
```

C3. The most serious mistake in track AI: data leakage

The one-line rule: SPLIT THE DATA FIRST, LEARN PARAMETERS AFTER

WRONG: fit the vectorizer on all the data, then split into train/test. The vectorizer has "seen" the test vocabulary, so the test score is higher than reality and meaningless.

RIGHT: split first. Fit the vectorizer on the train set ONLY. On the test set use transform only, never fit.

This rule applies to EVERYTHING that learns parameters from data: vectorizers, numeric scalers, imputers, feature selectors, dimensionality reducers.

Why this is an ethical problem, not just a technical bug: you report a number higher than the model's real ability. If someone trusts that number and deploys the model, the consequences are yours. This is the subject of question 4 of the final exam.

src/data/split.py and src/features/text.py — the right order

```
# ---------------- src/data/split.py ----------------
from sklearn.model_selection import train_test_split

def chia_du_lieu(df, nhan='issue_category', hat_giong=42):
    """Split 70/15/15, KEEPING the label ratio in all three sets (stratify)."""
    train, tam = train_test_split(
        df, test_size=0.30, random_state=hat_giong, stratify=df[nhan])
    val, test = train_test_split(
        tam, test_size=0.50, random_state=hat_giong, stratify=tam[nhan])
    return train, val, test

# ---------------- src/features/text.py ----------------
import re
from sklearn.feature_extraction.text import TfidfVectorizer

def lam_sach(s):
    """KEEP Vietnamese diacritics. Remove identifying data. Normalise spaces."""
    s = str(s).lower()
    s = re.sub(r'\b0\d{9}\b', ' <SDT> ', s)          # phone number
    s = re.sub(r'\bSN-?\d+\b', ' <SERIAL> ', s, flags=re.I)
    s = re.sub(r'\S+@\S+', ' <EMAIL> ', s)
    s = re.sub(r'[^\w\sàáảãạăằắẳẵặâầấẩẫậèéẻẽẹêềếểễệ'
               r'ìíỉĩịòóỏõọôồốổỗộơờớởỡợùúủũụưừứửữự'
               r'ỳýỷỹỵđ<>]', ' ', s)
    return re.sub(r'\s+', ' ', s).strip()

def tao_vectorizer():
    return TfidfVectorizer(ngram_range=(1, 2), min_df=2, max_features=20000)

# ---------------- THE RIGHT ORDER in train.py ----------------
# 1. load raw data
# 2. SPLIT train / val / test                    <-- done FIRST
# 3. clean the text of each set (learns no parameters, so it is safe)
# 4. vec = tao_vectorizer(); X_train = vec.fit_transform(train_text)
# 5. X_val = vec.transform(val_text);  X_test = vec.transform(test_text)
#    NEVER call fit or fit_transform on val or test
```

Why Vietnamese diacritics must be KEPT

Removing diacritics destroys meaning and merges different words: "sạc" (charge) and "sách" (book) both become "sach"; "lỗi" (fault) and "lối" (path) both become "loi".

In the Mekong Mobile data many descriptions differ only by diacritics, e.g. "màn hình" and "man hinh" (screen) — so lower-casing is fine, stripping diacritics loses information.

If you decide to strip diacritics for a concrete reason (e.g. 40% of the source text is written without them), state that reason in the report and MEASURE both options to justify the choice. That is traceability.

Illustration: the wrong order and the right order

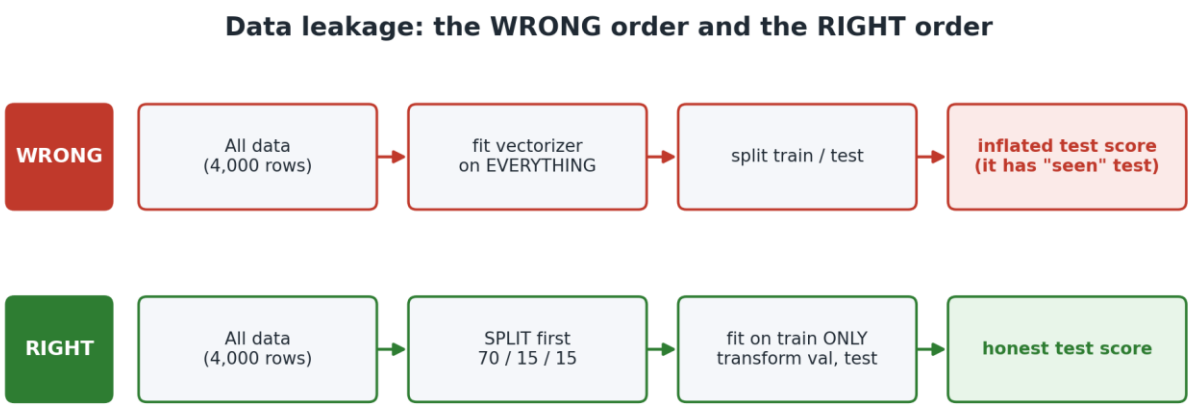


Figure 4. The only difference between the two rows is WHERE the split happens. The wrong order usually scores a few points higher on test — which is exactly why it is dangerous: a nicer number that is not true.

Example: three tests for the text-cleaning function

tests/test_text.py

```
# tests/test_text.py — run with: pytest -q
from src.features.text import lam_sach


def test_TT01_giu_dau_va_ve_chu_thuong():            # happy path
    assert lam_sach('Màn Hình bị SỌC') == 'màn hình bị sọc'


def test_TT02_thay_thong_tin_dinh_danh_bang_the():    # protects personal data
    s = lam_sach('Gọi 0912345678, máy SN-10293, mail an@vd.vn')
    assert '<SDT>' in s and '<SERIAL>' in s and '<EMAIL>' in s
    assert '0912345678' not in s


def test_TT03_chuoi_rong_hoac_chi_co_khoang_trang():  # invalid data: no exception
    assert lam_sach('') == ''
    assert lam_sach('   \n\t ') == ''
```

C4. A baseline model is mandatory

A baseline is the simplest possible model, used as a reference point. Without a baseline your number means nothing: 78% can be very good or very bad depending on what random guessing gives.

src/models/baseline.py — the two mandatory baselines

```
from sklearn.dummy import DummyClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, f1_score


def baseline_da_so(y_train, y_test):
    """Floor: always predict the most frequent label."""
    m = DummyClassifier(strategy='most_frequent').fit([[0]] * len(y_train), y_train)
```

```
    y_pred = m.predict([[0]] * len(y_test))
    return f1_score(y_test, y_pred, average='macro', zero_division=0)

def baseline_tfidf_logistic(X_train, y_train, X_test, y_test):
    """Standard: TF-IDF + logistic regression. Very hard to beat with complex models
       on a few thousand rows - measure it before trusting a big model."""
    m = LogisticRegression(max_iter=1000, class_weight='balanced').fit(X_train, y_train)
    y_pred = m.predict(X_test)
    print(classification_report(y_test, y_pred, zero_division=0))
    return f1_score(y_test, y_pred, average='macro', zero_division=0)
```

Three numbers that must be in reports/metrics.json for Assignment 2

1. Macro F1 of the majority-label model. This is the floor. Your model must beat it, otherwise it has learned nothing.

2. Macro F1 of TF-IDF + logistic regression. This is the standard. With a few thousand text rows it is usually very hard to beat.

3. Macro F1 of your chosen model, plus the F1 of each label. An overall score hides a model that completely ignores a rare label — report per label.

Why macro F1 rather than accuracy: accuracy rewards guessing the majority label. Macro F1 averages all labels equally, so it reveals when the model ignores rare labels. Be ready to answer this in session 8.

C5. Making re-runs give identical results

Four things to do for reproducible results

```
# 1. Put the random seed in the config, do NOT rely on defaults
#    configs/train.yaml:  hat_giong: 42
import random, numpy as np
random.seed(cfg['hat_giong']); np.random.seed(cfg['hat_giong'])

# 2. Pass the seed to EVERY place that uses randomness
train_test_split(..., random_state=cfg['hat_giong'])
LogisticRegression(random_state=cfg['hat_giong'])

# 3. Pin EXACT library versions in requirements.txt
#    scikit-learn==1.5.2      <-- use ==, never >=
#    pandas==2.2.3

# 4. Record the FULL MEASUREMENT CONDITIONS in metrics.json, not just the numbers
{
  "hat_giong": 42,
  "so_mau_train": 2800, "so_mau_val": 600, "so_mau_test": 600,
  "phien_ban_du_lieu": "issue_descriptions.csv, 4000 rows, seed 20260901",
  "f1_macro_baseline_da_so": 0.052,
  "f1_macro_baseline_tfidf_logistic": 0.741,
  "f1_macro_mo_hinh_chinh": 0.768,
  "f1_theo_nhan": { "MAN_HINH": 0.83, "PIN": 0.79, "SAC": 0.74,
                    "PHAN_MEM": 0.71, "NUOC_VAO": 0.80, "KHAC": 0.74 }
}

# The test: run training twice; the two metrics.json files must be IDENTICAL.
#   python -m src.models.train --config configs/train.yaml
#   cp reports/metrics.json /tmp/lan1.json
#   python -m src.models.train --config configs/train.yaml
#   diff /tmp/lan1.json reports/metrics.json      # must show no difference
```

The F1 numbers above only illustrate the format; they are not expected results. Your real results depend on how you clean the text and choose features.

Example: a train.py skeleton — one command from raw data to metrics.json

src/models/train.py

```
# src/models/train.py — run with: python -m src.models.train --config configs/train.yaml
import argparse, json, random, yaml, numpy as np, pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import f1_score
from src.data.split import chia_du_lieu
from src.features.text import lam_sach, tao_vectorizer
from src.models.baseline import baseline_da_so, baseline_tfidf_logistic

def main():
    ap = argparse.ArgumentParser(); ap.add_argument('--config', required=True)
    cfg = yaml.safe_load(open(ap.parse_args().config, encoding='utf-8'))
    hg = cfg['hat_giong']; random.seed(hg); np.random.seed(hg)

    df = pd.read_csv(cfg['duong_dan_du_lieu'])                    # 1. load
    train, val, test = chia_du_lieu(df, hat_giong=hg)             # 2. SPLIT FIRST
    t_tr, t_va, t_te = (x['issue_desc'].map(lam_sach) for x in (train, val, test))
    vec = tao_vectorizer()
    X_tr = vec.fit_transform(t_tr)                                # 3. fit on train ONLY
    X_va, X_te = vec.transform(t_va), vec.transform(t_te)
    y_tr, y_va, y_te = train['issue_category'], val['issue_category'], test['issue_category']

    # 4. choose C on VALIDATION; measure test ONCE, at the end
    def f1_val(C):
        m = LogisticRegression(C=C, max_iter=1000, class_weight='balanced', random_state=hg)
        return f1_score(y_va, m.fit(X_tr, y_tr).predict(X_va), average='macro')
    tot = max(cfg['C_thu'], key=f1_val)
    m = LogisticRegression(C=tot, max_iter=1000, class_weight='balanced',
                           random_state=hg).fit(X_tr, y_tr)
    y_pred = m.predict(X_te)
    ket_qua = {
        'hat_giong': hg, 'C_chon': tot,
        'so_mau_train': len(train), 'so_mau_val': len(val), 'so_mau_test': len(test),
        'f1_macro_baseline_da_so': round(baseline_da_so(y_tr, y_te), 3),
        'f1_macro_baseline_tfidf_logistic': round(
            baseline_tfidf_logistic(X_tr, y_tr, X_te, y_te), 3),
        'f1_macro_mo_hinh_chinh': round(f1_score(y_te, y_pred, average='macro'), 3),
        'f1_theo_nhan': {n: round(float(v), 3) for n, v in zip(m.classes_,
                         f1_score(y_te, y_pred, average=None, labels=m.classes_))},
    }
    with open('reports/metrics.json', 'w', encoding='utf-8') as f:
        json.dump(ket_qua, f, ensure_ascii=False, indent=2, sort_keys=True)  # stable diff

if __name__ == '__main__':
    main()
```

Read this skeleton through its four numbered steps

Step 2 comes before step 3 — that is section C3 written as code. The hyperparameter C is chosen on the validation set (step 4); the test set is used once, for reporting. configs/train.yaml needs three keys: hat_giong, duong_dan_du_lieu, C_thu (e.g. [0.3, 1, 3]).

C6. Self-check list before the session 8 workshop [AI]

- ☐ You have looked at the label distribution and can answer the three questions in C2.
- ☐ Data is split BEFORE fitting the vectorizer — recheck the order in train.py.
- ☐ No fit or fit_transform call runs on val or test.
- ☐ A training script runs with one command and produces reports/metrics.json.
- ☐ The reproducibility test is done: two runs give two identical metrics.json files.
- ☐ Both baselines exist and both numbers are in metrics.json.
- ☐ metrics.json has per-label F1, not only an overall score.

- ☐ The text-cleaning function keeps Vietnamese diacritics and replaces identifying data with tags.
- ☐ requirements.txt pins exact versions with ==.
- ☐ At least three tests for the text-cleaning function, passing with one command.

Quick comparison of the three tracks — a self-check table

Criterion	Track SE	Track DA	Track AI
What the working product is	The app server starts and the CRUD endpoints can be called	The pipeline runs with one command and produces staging and reject tables	The training script runs with one command and produces reports/metrics.json
The three separate layers	routes / services / repositories	extract / transform / load	data / features / models
Which layer holds the most important rules	services	transform and quality	features and models
Which layer needs the most testing	services (with fake repositories)	rules.py and checks.py (pure functions)	text.py (cleaning function)
The "re-runnable" checklist item	Migrations run a second time without errors	A second run does not duplicate records	Two runs give identical metrics.json
The most serious mistake of the track	Mixed layers: the service knows about HTTP	Silently dropping rows, no reject table	Data leakage: fitting the vectorizer before splitting
The question most often asked in session 8	"Which layer stops a malformed request?"	"What happens to the record count on a second run?"	"Why do you use macro F1 instead of accuracy?"
The shared principle for all three	Traceable · Reproducible · Verifiable	Traceable · Reproducible · Verifiable	Traceable · Reproducible · Verifiable

Three things to do this week, before the session 8 workshop

1. Build the folder skeleton following your track's template and commit it, even with empty files. With the skeleton in place, code goes into the right place and you never restructure later.

2. Write and run ONE test for your most important business rule. One running test is worth more than ten half-written ones.

3. Run your track's "re-runnable" test and write the result down. If it fails, that is your number-one job in session 8.

PART D — Shared by all tracks: commits, standard questions and troubleshooting

D1. Commit messages: weak and good examples

The course uses the Conventional Commits convention: <type>: <what was done>. Common types: feat (feature), fix (bug fix), test (tests), docs (documentation), refactor (restructure code without changing behaviour), chore (housekeeping). The commit history is evidence for CLO6.

Weak	Good	Why
update	feat: add POST /tickets endpoint with validator	You know what it contains without opening the diff.
fix bug	fix: QT-09 allow a new ticket once the old one is HUY	Names the rule and the behaviour fixed — traceable to the SRS.
finished part 2	test: add TC-02 warranty-expiry boundary	One commit, one job; easy to revert if this test breaks something.
final final v2	docs: README adds migration and seed commands	Documentation commits stay separate from code commits.

D2. The three standard questions applied to code — sample answers

Question	Weak answer	Good answer (SE example)
1. Why this approach rather than another?	"Because that is how it is done online."	"The service throws an AppError with a business code instead of an HTTP status, so the same rule serves the API, scripts and tests; routes is the only place that knows HTTP."
2. If the data grew tenfold, what breaks first?	"Probably nothing."	"findBySerial stays fast thanks to the idx_tickets_serial index. The weak spot is nextTicketCode counting COUNT(*) per centre — it slows down and can produce duplicate codes when two users create tickets at once; it should move to a SEQUENCE."
3. Show me the line of code responsible for this.	A vague verbal explanation.	Opens ticket.service.js and points at existing.find(...) and the throw new AppError('PHIEU_DANG_MO', ...) line of QT-09.

D3. Common problems when building the core module and quick fixes

Symptom	Usual cause	Fix
ECONNREFUSED 127.0.0.1:5432	PostgreSQL not running, or wrong port/host in .env	Start the DB; check DATABASE_URL; inside docker-compose use the service name (db) instead of localhost.
relation "tickets" already exists	Migration lacks IF NOT EXISTS	Add IF NOT EXISTS to every CREATE TABLE/INDEX, or use a migration tool with a version table.
ModuleNotFoundError: No module named 'src'	Running python src/pipeline.py instead of as a module	Run from the project root: python -m src.pipeline; add __init__.py to src/ and its subfolders.

Symptom	Usual cause	Fix
Tests pass on your machine, FAIL on the grader's	Tests depend on data already in your DB, or on the current date and time	Use fake repositories or fixtures that build their own data; pass a now parameter into date comparisons (as isUnderWarranty does).
Two training runs give different metrics.json	A missing random_state somewhere, or unpinned libraries	Pass hat_giong to EVERY random component; pin versions with ==; use sort_keys when writing JSON.
Pushed, but tag v0.2 is not on GitHub	git push origin main does not push tags	Also run git push origin v0.2, then open the releases/tag/v0.2 page to check.